

Definitions:

The best approximate eigenvectors and corresponding eigenvalues are extracted from the **Krylov subspace**

$$\mathcal{K}_k(A, \mathbf{v}) := \text{span}\{\mathbf{v}, A\mathbf{v}, A^2\mathbf{v}, \dots, A^{k-1}\mathbf{v}\}.$$

The approximate eigenpairs are constructed through a Galerkin condition. An approximate eigenvector $\mathbf{x} \in \mathcal{S}$ is called a **Ritz vector** with corresponding **Ritz value** θ if the **Galerkin condition**

$$\mathbf{w}^*(A\mathbf{x} - \mathbf{x}\theta) = 0, \text{ for all } \mathbf{w} \in \mathcal{K}_k(A, \mathbf{v})$$

is satisfied.

Facts: [Sor92], [Sor02]

1. Every $\mathbf{w} \in \mathcal{K}_k$ is of the form $\mathbf{w} = \phi(A)\mathbf{v}_1$ for some polynomial ϕ of degree less than k and $\mathcal{K}_{j-1} \subset \mathcal{K}_j$ for $j = 2, 3, \dots, k$.
2. If a sequence of orthogonal bases $V_k = [\mathbf{v}_1, \mathbf{v}_2, \dots, \mathbf{v}_k]$ has been constructed with $\mathcal{K}_k = \text{range}(V_k)$ and $V_k^* V_k = I_k$, then a new basis vector $\mathbf{v}_{k+1}$ is obtained by the **projection formulas**

$$\mathbf{h}_k = V_k^* A \mathbf{v}_k,$$

$$\mathbf{f}_k = A \mathbf{v}_k - V_k \mathbf{h}_k,$$

$$\mathbf{v}_{k+1} = \mathbf{f}_k / \|\mathbf{f}_k\|_2.$$

The vector $\mathbf{h}_k$ is constructed to achieve $V_k^* \mathbf{f}_k = 0$ so that $\mathbf{v}_{k+1}$ is a vector of unit length that is orthogonal to the columns of V_k .

3. The columns of $V_{k+1} = [V_k, \mathbf{v}_{k+1}]$ provide an orthonormal basis for $\mathcal{K}_{k+1}(A, \mathbf{v}_1)$.
4. The basis vectors are of the form $\mathbf{v}_j = \phi_{j-1}(A)\mathbf{v}_1$, where ϕ_{j-1} is a polynomial of degree $j - 1$ for each $j = 1, 2, \dots, k + 1$.
5. This construction fails when $\mathbf{f}_k = 0$, but then

$$A V_k = V_k H_k,$$

where $H_k = V_k^* A V_k = [\mathbf{h}_1, \mathbf{h}_2, \dots, \mathbf{h}_k]$ (with a slight abuse of notation). This “good breakdown” happens precisely when $\mathcal{K}_k$ is an invariant subspace of A . Hence, $\sigma(H_k) \subset \sigma(A)$.

44.2 The Arnoldi Factorization

The projection formulas given above result in the fundamental Arnoldi method for constructing an orthonormal basis for $\mathcal{K}_k$.

Definitions:

The relations between the matrix A , the basis matrix V_k and the residual vector $\mathbf{f}_k$ may be concisely expressed as

$$A V_k = V_k H_k + \mathbf{f}_k \mathbf{e}_k^*,$$

where $V_k \in \mathbb{C}^{n \times k}$ has orthonormal columns, $V_k^* \mathbf{f}_k = 0$, and $H_k = V_k^* A V_k$ is a $k \times k$ upper Hessenberg matrix with nonnegative subdiagonal elements.

The above expression shall be called a **k-step Arnoldi factorization** of A . When A is Hermitian, H_k will be real, symmetric, and tridiagonal and then the relation is called a **k-step Lanczos factorization** of A . The columns of V_k are referred to as **Arnoldi vectors** or **Lanczos vectors**, respectively. The Hessenberg matrix H_k is called **unreduced** if all subdiagonal elements are nonzero.

Facts: [Sor92], [Sor02]

1. The explicit steps needed to form a k -step Arnoldi factorization are shown in Algorithm 1.

Algorithm 1: k -step Arnoldi factorization. A square matrix A , a nonzero vector $\mathbf{v}$ and a positive integer $k \leq n$ are input.

Output is an $n \times k$ ortho-normal matrix V_k , an upper Hessenberg matrix H_k and a vector $\mathbf{f}_k$ such that $AV_k = V_k H_k + \mathbf{f}_k \mathbf{e}_k^T$.

```

 $\mathbf{v}_1 = \mathbf{v} / \|\mathbf{v}\|_2;$ 
 $\mathbf{w} = A\mathbf{v}_1; \alpha_1 = \mathbf{v}_1^* \mathbf{w};$ 
 $\mathbf{f}_1 \leftarrow \mathbf{w} - \mathbf{v}_1 \alpha_1;$ 
 $V_1 \leftarrow [\mathbf{v}_1]; H_1 \leftarrow [\alpha_1];$ 
for  $j = 1, 2, 3, \dots, k-1,$ 
     $\beta_j = \|\mathbf{f}_j\|_2; \mathbf{v}_{j+1} \leftarrow \mathbf{f}_j / \beta_j;$ 
     $V_{j+1} \leftarrow [V_j, \mathbf{v}_{j+1}];$ 

     $\hat{H}_j \leftarrow \begin{bmatrix} H_j \\ \beta_j \mathbf{e}_j^* \end{bmatrix};$ 

     $\mathbf{w} \leftarrow A\mathbf{v}_{j+1};$ 
     $\mathbf{h} \leftarrow V_{j+1}^* \mathbf{w};$ 
     $\mathbf{f}_{j+1} \leftarrow \mathbf{w} - V_{j+1} \mathbf{h};$ 
     $H_{j+1} \leftarrow [\hat{H}_j, \mathbf{h}];$ 
end

```

2. Ritz pairs satisfying the Galerkin condition (see Section 44.1) are derived from the eigenpairs of the small projected matrix H_k . If $H_k \mathbf{y} = \mathbf{y} \theta$ with $\|\mathbf{y}\|_2 = 1$, then the vector $\mathbf{x} = V_k \mathbf{y}$ is a vector of unit norm that satisfies

$$\|A\mathbf{x} - \mathbf{x}\theta\|_2 = \|(AV_k - V_k H_k) \mathbf{y}\|_2 = |\beta_k \mathbf{e}_k^* \mathbf{y}|,$$

where $\beta_k = \|\mathbf{f}_k\|_2$.

3. If $(\mathbf{x}, \theta)$ is a Ritz pair constructed as shown in Fact 2, then

$$\theta = \mathbf{y}^* H_k \mathbf{y} = (V_k \mathbf{y})^* A (V_k \mathbf{y}) = \mathbf{x}^* A \mathbf{x}$$

is always a Rayleigh quotient (assuming $\|\mathbf{y}\|_2 = 1$).

4. The Rayleigh quotient residual $\mathbf{r}(\mathbf{x}) := A\mathbf{x} - \mathbf{x}\theta$ satisfies $\|\mathbf{r}(\mathbf{x})\|_2 = |\beta_k \mathbf{e}_k^* \mathbf{y}|$. When A is Hermitian, this relation provides computable rigorous bounds on the accuracy of the approximate eigenvalues [Par80]. When A is non-Hermitian, one needs additional sensitivity information. Nonnormality effects may corrupt the accuracy. In exact arithmetic, these Ritz pairs are eigenpairs of A whenever $\mathbf{f}_k = 0$. However, even with a very small residual these may be far from actual eigenvalues when A is highly nonnormal.
5. The orthogonalization process is based upon the *classical Gram–Schmidt* (CGS) scheme. This process is notoriously unstable and will fail miserably in this application without modification.

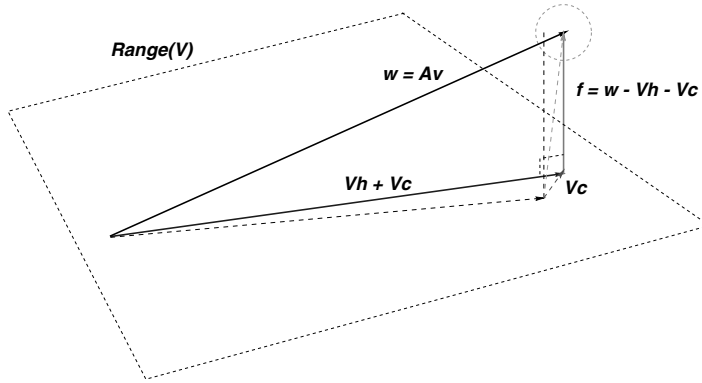


FIGURE 44.1 DGKS Correction.

The iterative refinement technique proposed by Daniel, Gragg, Kaufman, and Stewart (DGKS) [DGK76] provides an excellent way to construct a vector $\mathbf{f}_{j+1}$ that is numerically orthogonal to V_{j+1} . It amounts to computing a correction

$$\mathbf{c} = V_{j+1}^* \mathbf{f}_{j+1}; \quad \mathbf{f}_{j+1} \leftarrow \mathbf{f}_{j+1} - V_{j+1} \mathbf{c}; \quad \mathbf{h} \leftarrow \mathbf{h} + \mathbf{c};$$

just after computing $\mathbf{f}_{j+1}$ if necessary, i.e., when $\mathbf{f}_{j+1}$ is not sufficiently orthogonal to the columns of V_{j+1} . This formulation is crucial to both accuracy and performance. It provides numerically orthogonal basis vectors and it may be implemented using the Level 2 BLAS operation `_GEMV` [DDH88]. This provides a significant performance advantage on virtually every platform from workstation to supercomputer.

6. The *modified Gram-Schmidt* (MGS) process will generally fail to produce orthogonal vectors and cannot be implemented with Level 2 BLAS in this setting. ARPACK relies on a restarting scheme wherein the goal is to reach a state of dependence in order to obtain $\mathbf{f}_k = 0$. MGS is completely inappropriate for this situation, but the CGS with DGKS correction performs beautifully.
7. Failure to maintain orthogonality leads to numerical difficulties in the Lanczos/Arnoldi process. Loss of orthogonality typically results in the presence of spurious copies of the approximate eigenvalue.

Examples:

1. Figure 44.1 illustrates how the DGKS mechanism works. When the vector $\mathbf{w} = A\mathbf{v}$ is nearly in the $\text{range}(V)$ then the projection $V\mathbf{h}$ is possibly inaccurate, but vector $\mathbf{f} = \mathbf{w} - V\mathbf{h}$ is not close to $\text{range}(V)$ and can be safely orthogonalized to compute the correction $\mathbf{c}$ accurately. The corrected vector $\mathbf{f} \leftarrow \mathbf{f} - V\mathbf{c}$ will be numerically orthogonal to the columns of V in almost all cases. Additional corrections might be necessary in very unusual cases.

44.3 Restarting the Arnoldi Process

The number of Arnoldi steps required to calculate eigenvalues of interest to a specified accuracy cannot be pre-determined. Usually, eigen-information of interest does not appear until k gets very large. In Figure 44.2 the distribution in the complex plane of the Ritz values (shown in grey dots) is compared with the spectrum (shown as +s). The original matrix is a normally distributed random matrix of order 200 and the Ritz values are from a $(k = 50)$ -step Arnoldi factorization. Eigenvalues at the extremes of the spectrum of A are clearly better approximated than the interior eigenvalues.

For large problems, it is intractable to compute and store a numerically orthogonal basis set V_k for large k . Storage requirements are $\mathcal{O}(n \cdot k)$ and arithmetic costs are $\mathcal{O}(n \cdot k^2)$ flops to compute the basis vectors

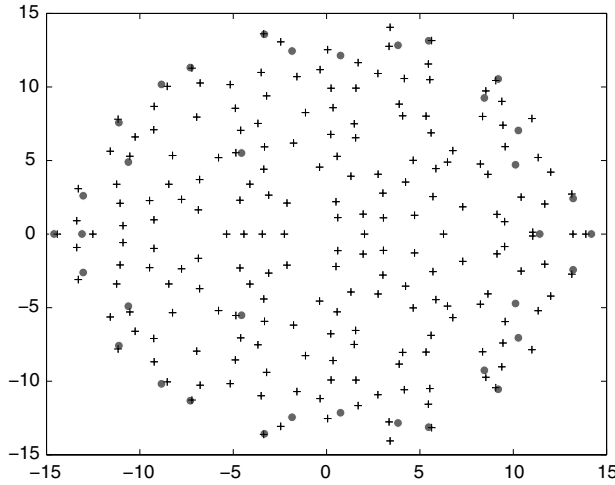


FIGURE 44.2 Typical distribution of Ritz values.

plus $\mathcal{O}(k^3)$ flops to compute the eigensystem of H_k . Thus, restarting schemes have been developed that iteratively replace the starting vector $\mathbf{v}_1$ with an “improved” starting vector $\mathbf{v}_1^+$ and then compute a new Arnoldi factorization of fixed length k to limit the costs. Beyond this, there is an interest in forcing $\mathbf{f}_k = 0$ and, thus, producing an invariant subspace. However, this is useful only if the spectrum $\sigma(H_k)$ has the desired properties.

The structure of $\mathbf{f}_k$ suggests the restarting strategy. The goal will be to iteratively force $\mathbf{v}_1$ to be a linear combination of eigenvectors of interest.

Facts: [Sor92], [Sor02]

1. If $\mathbf{v} = \sum_{j=1}^k \mathbf{q}_j \gamma_j$ where $A\mathbf{q}_j = \mathbf{q}_j \lambda_j$ and

$$A\mathbf{v} = V\mathbf{H} + \mathbf{f}_k^T$$

is a k -step Arnoldi factorization with unreduced H , then $\mathbf{f} = 0$ and $\sigma(H) = \{\lambda_1, \lambda_2, \dots, \lambda_k\}$.

2. Since $\mathbf{v}_1$ determines the subspace $\mathcal{K}_k$, this vector must be constructed to select the eigenvalues of interest. The starting vector must be forced to become a linear combination of eigenvectors that span the desired invariant subspace. There is a necessary and sufficient condition for $\mathbf{f}$ to vanish that involves Schur vectors and does not require diagonalizability.

44.4 Polynomial Restarting

Polynomial restarting strategies replace $\mathbf{v}_1$ by

$$\mathbf{v}_1 \leftarrow \psi(A)\mathbf{v}_1,$$

where ψ is a polynomial constructed to damp unwanted components from the starting vector. If $\mathbf{v}_1 = \sum_{j=1}^n \mathbf{q}_j \gamma_j$ where $A\mathbf{q}_j = \mathbf{q}_j \lambda_j$, then

$$\mathbf{v}_1^+ = \psi(A)\mathbf{v}_1 = \sum_{j=1}^n \mathbf{q}_j \gamma_j \psi(\lambda_j),$$

where the polynomial ψ has also been normalized to give $\|\mathbf{v}_1\|_2 = 1$. Motivated by the structure of $\mathbf{f}_k$, the idea is to force the starting vector to be closer and closer to an invariant subspace by constructing ψ so that $|\psi(\lambda)|$ is as small as possible on a region containing the unwanted eigenvalues.

An iteration is defined by repeatedly restarting until the updated Arnoldi factorization eventually contains the desired eigenspace. An explicit scheme for restarting was proposed by Saad in [Saa92]. One of the more successful choices is to use Chebyshev polynomials in order to damp unwanted eigenvector components.

Definitions:

The polynomial ψ is sometimes called a **filter polynomial**, which may also be specified by its roots. The roots of the filter polynomial may also be referred to as **shifts**. This terminology refers to their usage in an implicitly shifted QR-iteration. One straightforward choice of shifts is to find the eigenvalues θ_j of the projected matrix H and sort these into two sets according to a given criterion: the wanted set $\mathcal{W} = \{\theta_j : j = 1, 2, \dots, k\}$ and the unwanted set $\mathcal{U} = \{\theta_j : j = k+1, k+2, \dots, k+p\}$. Then one specifies the polynomial ψ as the polynomial with these unwanted Ritz values as its roots. This choice of roots, called **exact shifts**, was suggested in [Sor92].

Facts: [Sor92], [Sor02]

1. Morgan [Mor96] found a remarkable property of this strategy. If exact shifts are used to define $\psi(\tau) = \prod_{j=k+1}^{k+p} (\tau - \theta_j)$ and if $\hat{\mathbf{q}}_j$ denotes a Ritz vector of unit length corresponding to θ_j , then the Krylov space generated by $\mathbf{v}_1^+ = \psi(A)\mathbf{v}_1$ satisfies

$$\mathcal{K}_m(A, \mathbf{v}_1^+) = \text{Span}\{\hat{\mathbf{q}}_1, \hat{\mathbf{q}}_2, \dots, \hat{\mathbf{q}}_k, A\hat{\mathbf{q}}_j, A^2\hat{\mathbf{q}}_j, \dots, A^p\hat{\mathbf{q}}_j\},$$

for any $j = 1, 2, \dots, k$. Thus, polynomial restarting with exact shifts will generate a new subspace that contains all of the possible choices of updated starting vector consisting of linear combinations of the wanted Ritz vectors.

2. Exact shifts tend to perform remarkably well in practice and have been adopted as the shift selection of choice in ARPACK when no other information is available. However, there are many other possibilities such as the use of Leja points for certain containment regions or intervals [BCR96].

44.5 Implicit Restarting

There are a number of schemes used to implement polynomial restarting. We shall focus on an implicit restarting scheme.

Definitions:

A straightforward way to implement polynomial restarting is to explicitly construct the starting vector $\mathbf{v}_1^+ = \psi(A)\mathbf{v}_1$ by applying $\psi(A)$ through a sequence of matrix-vector products. This is called **explicit restarting**. A more efficient and numerically stable alternative is **implicit restarting**. This technique applies a sequence of implicitly shifted QR steps to an m -step Arnoldi or Lanczos factorization to obtain a truncated form of the implicitly shifted QR-iteration.

On convergence, the IRAM iteration (see Algorithm 2) gives an orthonormal matrix V_k and an upper Hessenberg matrix H_k such that $AV_k \approx V_k H_k$. If $H_k Q_k = Q_k R_k$ is a Schur decomposition of H_k , then we call $\hat{V}_k \equiv V_k Q_k$ a **Schur basis** for the Krylov subspace $\mathcal{K}_k(A, \mathbf{v}_1)$. Note that if $AV_k = V_k H_k$ exactly, then $\hat{V}_k$ would form the leading k columns of a unitary matrix $\hat{V}$ and R_k would form the leading $k \times k$ block of an upper triangular matrix R , where $A\hat{V} = \hat{V}R$ is a complete Schur decomposition. We refer to this as a **partial Schur decomposition** of A .

Algorithm 2: IRAM iteration

Input is an $n \times k$ ortho-normal matrix V_k , an upper Hessenberg matrix H_k and a vector $\mathbf{f}_k$ such that $AV_k = V_k H_k + \mathbf{f}_k \mathbf{e}_k^T$.

Output is an $n \times k$ ortho-normal matrix V_k , an upper triangular matrix H_k such that $AV_k = V_k H_k$.

repeat until convergence,

Beginning with the k -step factorization,
apply p additional steps of the Arnoldi process
to compute an $m = k + p$ step Arnoldi factorization

$$AV_m = V_m H_m + \mathbf{f}_m \mathbf{e}_m^*.$$

Compute $\sigma(H_m)$ and select p shifts $\mu_1, \mu_2, \dots, \mu_p$;

$$Q = I_m;$$

for $j = 1, 2, \dots, p$,

$$\text{Factor } [Q_j, R_j] = \text{qr}(H_m - \mu_j I);$$

$$H_m \leftarrow Q_j^* H_m Q_j;$$

$$Q \leftarrow Q Q_j;$$

end

$$\hat{\beta}_k = H_m(k+1, k); \sigma_k = Q(m, k);$$

$$\mathbf{f}_k \leftarrow \mathbf{v}_{k+1} \hat{\beta}_k + \mathbf{f}_m \sigma_k;$$

$$V_k \leftarrow V_m Q(:, 1:k); H_k \leftarrow H_m(1:k, 1:k);$$

end

Facts: [Sor92], [Sor02]

1. Implicit restarting avoids numerical difficulties and storage problems normally associated with Arnoldi and Lanczos processes. The algorithm is capable of computing a few (k) eigenvalues with user specified features such as largest real part or largest magnitude using $2nk + \mathcal{O}(k^2)$ storage. The computed Schur basis vectors for the desired k -dimensional eigenspace are numerically orthogonal to working precision.
2. Desired eigen-information from a high-dimensional Krylov space is continually compressed into a fixed size k -dimensional subspace through an implicitly shifted QR mechanism. An Arnoldi factorization of length $m = k + p$,

$$AV_m = V_m H_m + \mathbf{f}_m \mathbf{e}_m^*,$$

is compressed to a factorization of length k that retains the eigen-information of interest. Then the factorization is expanded once more to m -steps and the compression process is repeated.

3. QR steps are used to apply p linear polynomial factors $A - \mu_j I$ implicitly to the starting vector $\mathbf{v}_1$. The first stage of this shift process results in

$$AV_m^+ = V_m^+ H_m^+ + \mathbf{f}_m \mathbf{e}_m^* Q,$$

where $V_m^+ = V_m Q$, $H_m^+ = Q^* H_m Q$, and $Q = Q_1 Q_2 \dots Q_p$. Each Q_j is the orthogonal matrix associated with implicit application of the shift $\mu_j = \theta_{k+j}$. Since each of the matrices Q_j is Hessenberg, it turns out that the first $k-1$ entries of the vector $\mathbf{e}_m^* Q$ are zero (i.e., $\mathbf{e}_m^* Q = [\sigma \mathbf{e}_k^T, \hat{\mathbf{q}}^*]$). Hence, the leading k columns remain in an Arnoldi relation and provide an updated k -step Arnoldi factorization

$$AV_k^+ = V_k^+ H_k^+ + \mathbf{f}_k^+ \mathbf{e}_k^*,$$

with an updated residual of the form $\mathbf{f}_k^+ = V_m^+ \mathbf{e}_{k+1} \hat{\beta}_k + \mathbf{f}_m \sigma$. Using this as a starting point, it is possible to apply p additional steps of the Arnoldi process to return to the original m -step form.

4. Virtually any explicit polynomial restarting scheme can be applied with implicit restarting, but considerable success has been obtained with exact shifts. Exact shifts result in H_k^+ having the k wanted Ritz values as its spectrum. As convergence takes place, the subdiagonals of H_k tend to zero and the most desired eigenvalue approximations appear as eigenvalues of the leading $k \times k$ block of R as a partial Schur decomposition of A . The basis vectors V_k tend to numerically orthogonal Schur vectors.
5. The basic IRAM iteration is shown in Algorithm 2.

Examples:

1. The expansion and contraction process of the IRAM iteration is visualized in Figure 44.3.

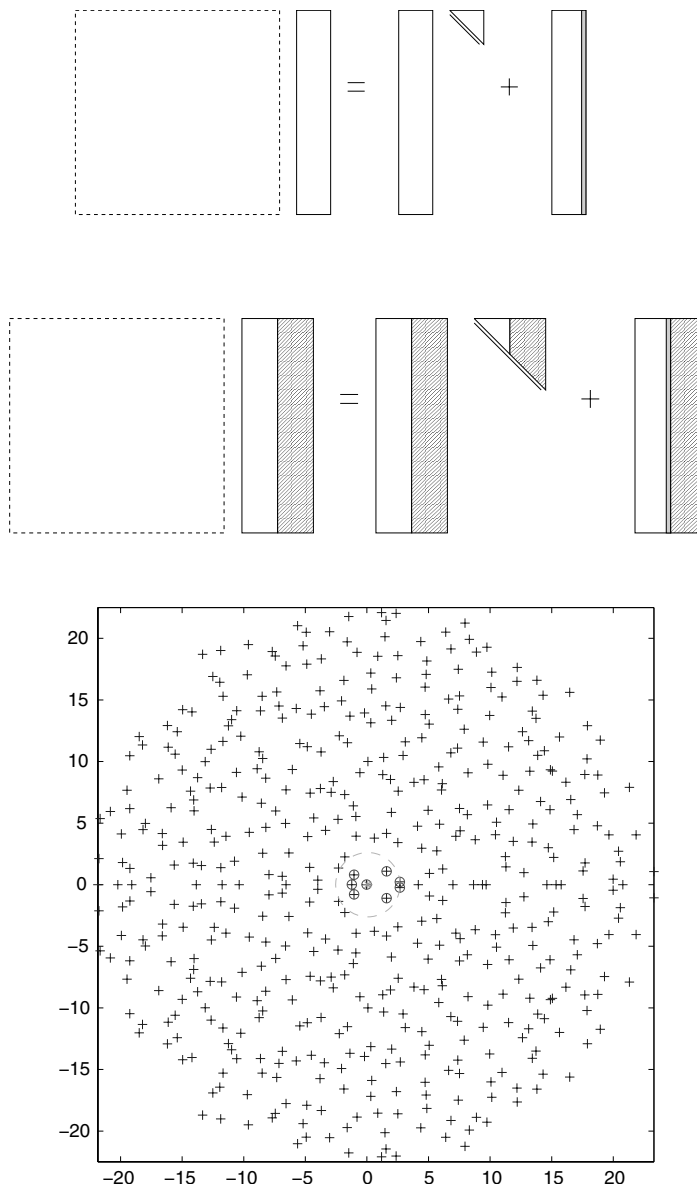


FIGURE 44.3 Visualization of IRAM.

44.6 Convergence of IRAM

IRAM converges linearly. An intuitive explanation follows. If $\mathbf{v}_1$ is expressed as a linear combination of eigenvectors $\{\mathbf{q}_j\}$ of A , then

$$\mathbf{v}_1 = \sum_{j=1}^n \mathbf{q}_j \gamma_j \Rightarrow \psi(A)\mathbf{v}_1 = \sum_{j=1}^n \mathbf{q}_j \psi(\lambda_j) \gamma_j.$$

Applying the same polynomial (i.e., using the same shifts) repeatedly for ℓ iterations will result in the j -th original expansion coefficient being attenuated by a factor

$$\left(\frac{\psi(\lambda_j)}{\psi(\lambda_1)} \right)^\ell,$$

where the eigenvalues have been ordered according to decreasing values of $|\psi(\lambda_j)|$. The leading k eigenvalues become dominant in this expansion and the remaining eigenvalues become less and less significant as the iteration proceeds. Hence, the starting vector $\mathbf{v}_1$ is forced into an invariant subspace as desired. The adaptive choice of ψ provided with the exact shift mechanism further enhances the isolation of the wanted components in this expansion. Hence, the wanted eigenvalues are approximated ever better as the iteration proceeds. Making this heuristic argument precise has turned out to be quite difficult. Some fairly sophisticated analysis is required to understand convergence of these methods.

44.7 Convergence in Gap: Distance to a Subspace

To fully discuss convergence we need some notion of nearness of subspaces. When nonnormality is present or when eigenvalues are clustered, the distance between the computed subspace and the desired subspace is a better measure of success than distance between eigenvalues. The subspaces carry uniquely defined Ritz values with them, but these can be very sensitive to perturbations in the nonnormal setting.

Definitions:

A notion of distance that is useful in our setting is the **containment gap** between the subspaces $\mathcal{W}$ and $\mathcal{V}$:

$$\delta(\mathcal{W}, \mathcal{V}) := \max_{\mathbf{w} \in \mathcal{W}} \min_{\mathbf{v} \in \mathcal{V}} \frac{\|\mathbf{w} - \mathbf{v}\|_2}{\|\mathbf{w}\|_2}.$$

Note: $\delta(\mathcal{W}, \mathcal{V})$ is the sine of the largest canonical angle between $\mathcal{W}$ and the closest subspace of $\mathcal{V}$ with the same dimension as $\mathcal{W}$.

In keeping with the terminology developed in [BER04] and [BES05], $\mathcal{X}_g$ shall be the invariant subspace of A associated with the so called “good” eigenvalues (the desired eigenvalues) and $\mathcal{X}_b$ is the complementary subspace. $\mathbf{P}_g$ and $\mathbf{P}_b$ are the spectral projectors with respect to these spaces.

It is desirable to have **convergence in Gap** for the Krylov method, meaning

$$\delta(\mathcal{K}_m(A, \mathbf{v}_1^{(\ell)}), \mathcal{X}_g) \rightarrow 0.$$

Fundamental quantities required to study convergence.

1. Minimal Polynomial for $\mathcal{X}_g$:

$$a_g := \text{minimal polynomial of } A \text{ with respect to } \mathbf{P}_g \mathbf{v}_1,$$

which is the monic polynomial of least degree s.t. $a_g(A)\mathbf{P}_g \mathbf{v}_1 = \mathbf{0}$.

2. **Nonnormality constant $\kappa(\Omega)$** : The smallest positive number s.t.

$$\|f(A) \Pi_{\mathcal{U}}\|_2 \leq \kappa(\Omega) \max_{z \in \Omega} |f(z)|$$

uniformly for all functions f analytic on Ω . This constant and its historical origins are discussed in detail in [BER04].

3. **ε -pseudospectrum of A** :

$$\Lambda_\varepsilon(A) := \{z \in \mathbf{C} : \|(zI - A)^{-1}\|_2 \geq \varepsilon^{-1}\}.$$

Facts: [BER04], [BES05]

1. Two fundamental convergence questions:

- What is the gap $\delta(\mathcal{U}_g, \mathcal{K}_k(A, \mathbf{v}_1))$ as k increases?
- How does $\delta(\mathcal{U}_g, \mathcal{K}_m(A, \widehat{\mathbf{v}}_1))$ depend on $\widehat{\mathbf{v}}_1 = \Phi(A)\mathbf{v}_1$, and how can we optimize the asymptotic behavior?

Key ingredients to convergence behavior are the nonnormality of A and the distribution of $\mathbf{v}_1$ w. r. t. $\mathcal{U}_g$. The goal of restarting is to attain the unrestarted iteration performance, but within **restricted subspace dimensions**.

2. **Convergence with no restarts:** In [BES05], it is shown that

$$\delta(\mathcal{U}_g, \mathcal{K}_\ell(A, \mathbf{v}_1)) \leq C_o C_b \min_{p \in \mathcal{P}_{\ell-2m}} \max_{z \in \Omega_b} |1 - a_g(z)p(z)|,$$

where the compact set $\Omega_g \subseteq \mathbf{C} \setminus \Omega_b$ contains all the good eigenvalues.

$$C_o := \max_{\psi \in \mathcal{P}_{m-1}} \frac{\|\psi(A)\mathbf{P}_b\mathbf{v}_1\|_2}{\|\psi(A)\mathbf{P}_g\mathbf{v}_1\|_2}, \quad C_b := \kappa(\Omega_b).$$

3. Rate of convergence estimates are obtained from complex approximation theory. Construct conformal map $\mathcal{G}$ taking the exterior of Ω_b to the exterior of the unit disk with $\mathcal{G}(\infty) = \infty$ and $\mathcal{G}'(\infty) > 0$. Define $\rho := (\min_{j=1,\dots,L} |\mathcal{G}(\lambda_j)|)^{-1}$. Then (Gaier, Walsh)

$$\limsup_{k \rightarrow \infty} \min_{p \in \mathcal{P}_k} \max_{z \in \Omega_b} \left| \frac{1}{a_g(z)} - p(z) \right|^{1/k} = \rho.$$

The image of $\{|z| = \rho^{-1}\}$ is a curve $\mathcal{C} := \mathcal{G}^{-1}(\{|z| = \rho^{-1}\})$ around Ω_b . This critical curve passes through a good eigenvalue “closest to” Ω_b . The curve contains at least one good eigenvalue, with all bad and no good eigenvalues in its interior.

4. Convergence with the exact shift strategy has not yet been fully analyzed. However, convergence rates have been established for restarts with asymptotically optimal points. These are the Fejér, Fekete, or Leja points for Ω_b . In [BES05], computational experiments are shown that indicate that exact shifts behave very much like optimal points for certain regions bounded by pseudo-spectral level curves or lemniscates.
5. Let Ψ_M interpolate $1/a_g(z)$ at the M restart shifts:

$$\delta(\mathcal{U}_g, \mathcal{K}_\ell(A, \widehat{\mathbf{v}}_1)) \leq C_o C_g \max_{z \in \Omega_b} |1 - \Psi_M(z)a_g(z)| \leq C_o C_g C_r r^M$$

for any $r > \rho$ (see [Gai87], [FR89]). Here, $\widehat{\mathbf{v}}_1 = \Phi(A)\mathbf{v}_1$, where Φ is the aggregate restart polynomial (its roots are all the implicit restart shifts that have been applied). The subspace dimension is $\ell = 2m$, the restart degree is m , and the aggregate degree is $M = \nu m$.

44.8 The Generalized Eigenproblem

In many applications, the generalized eigenproblem $A\mathbf{x} = M\mathbf{x}\lambda$ arises naturally. A typical setting is a finite element discretization of a continuous problem where the matrix M arises from inner products of basis functions. In this case, M is symmetric and positive (semi) definite, and for some algorithms this property is a necessary condition. Generally, algorithms are based upon transforming the generalized problem to a standard problem.

44.9 Krylov Methods with Spectral Transformations

Definitions:

A very successful scheme for converting the generalized problem to a standard problem that is amenable to a Krylov or a subspace iteration method is to use the **spectral transformation** suggested by Ericsson and Ruhe [ER80],

$$(A - \sigma M)^{-1} M \mathbf{x} = \nu \mathbf{x}.$$

Facts: [Sor92], [Sor02]

1. An eigenvector $\mathbf{x}$ of the spectral transformation is also an eigenvector of the original problem $A\mathbf{x} = M\mathbf{x}\lambda$, with the corresponding eigenvalue given by $\lambda = \sigma + \frac{1}{\nu}$.
2. There is generally rapid convergence to eigenvalues near the shift σ because they are transformed to extremal well-separated eigenvalues. Perhaps an even more influential aspect of this transformation is that eigenvalues far from σ are damped (mapped near zero).
3. One strategy is to choose σ to be a point in the complex plane that is near eigenvalues of interest and then compute the eigenvalues ν of largest magnitude of the spectral transformation matrix. It is not necessary to have σ extremely close to an eigenvalue. This transformation together with the implicit restarting technique is usually adequate for computing a significant number of eigenvalues near σ .
4. Even when $M = I$, one generally must use the shift-invert spectral transformation to find interior eigenvalues. The extreme eigenvalues of the transformed operator A_σ are generally large and well separated from the rest of the spectrum. The eigenvalues ν of largest magnitude will transform back to eigenvalues λ of the original A that are in a disk about the point σ . This is illustrated in Figure 44.4, where the $+$ symbols are the eigenvalues of A and the circled ones are the computed eigenvalues in the disk (dashed circle) centered at the point σ .
5. With shift-invert, the Arnoldi process is applied to the matrix $A_\sigma := (A - \sigma M)^{-1} M$. Whenever a matrix-vector product $\mathbf{w} \leftarrow A_\sigma \mathbf{v}$ is required, the following steps are performed:
 - $\mathbf{z} = M\mathbf{v}$,
 - Solve $(A - \sigma M)\mathbf{w} = \mathbf{z}$ for $\mathbf{w}$.

The matrix $A - \sigma M$ is factored initially with a sparse direct LU-decomposition or in a symmetric indefinite factorization and this single factorization is used repeatedly to apply the matrix operator A_σ as required.

6. The scheme is modified to preserve symmetry when A and M are both symmetric and M is positive (semi)definite. One can utilize a weighted M (semi) inner product in the Lanczos/Arnoldi process [ER80], [GLS94], [MS97]. This amounts to replacing the computation of $\mathbf{h} \leftarrow V_{j+1}^* \mathbf{w}$ and $\beta_j = \|\mathbf{f}_j\|_2$ with $\mathbf{h} \leftarrow V_{j+1}^* M \mathbf{w}$ and $\beta_j = \sqrt{\mathbf{f}_j^* M \mathbf{f}_j}$, respectively, in the Arnoldi process described in Algorithm 1.
7. The matrix operator A_σ is self-adjoint with respect to this (semi)inner product, i.e., $\langle A_\sigma \mathbf{x}, \mathbf{y} \rangle = \langle \mathbf{x}, A_\sigma \mathbf{y} \rangle$ for all vectors $\mathbf{x}, \mathbf{y}$, where $\langle \mathbf{w}, \mathbf{v} \rangle := \sqrt{\mathbf{w}^* M \mathbf{v}}$. This implies that the projected Hessenberg

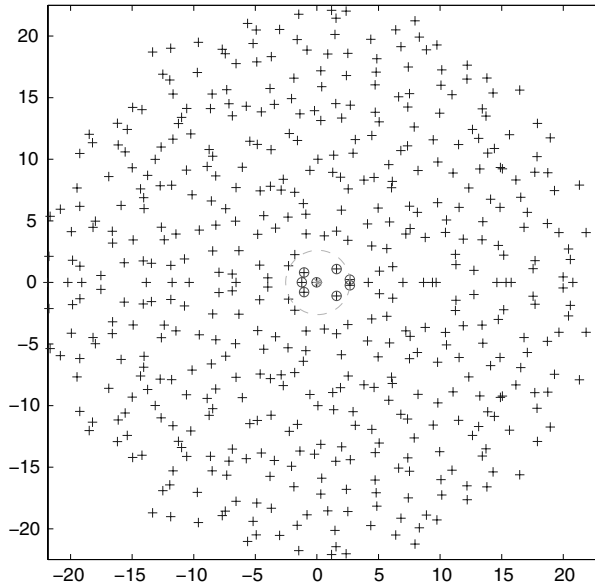


FIGURE 44.4 Eigenvalues from shift-invert.

matrix H is actually symmetric and tridiagonal and the standard three-term Lanczos recurrence is recovered with this inner product.

8. There is a subtle aspect to this approach when M is singular. The most pathological case, when $\text{null}(A) \cap \text{null}(M) \neq \{0\}$, is not treated here. However, when M is singular there may be infinite eigenvalues of the pair (A, M) and the presence of these can introduce large perturbations to the computed Ritz values and vectors. To avoid these difficulties, a purging operation has been suggested by Ericsson and Ruhe [ER80]. If $\mathbf{x} = V\mathbf{y}$ with $H\mathbf{y} = \mathbf{y}\theta$, then

$$A_\sigma \mathbf{x} = V H \mathbf{y} + \mathbf{f} \mathbf{e}_k^T \mathbf{y} = \mathbf{x} \theta + \mathbf{f} \mathbf{e}_k^T \mathbf{y}.$$

Replacing the $\mathbf{x}$ with the improved eigenvector approximation $\mathbf{x} \leftarrow (\mathbf{x} + \frac{1}{\theta} \mathbf{f} \mathbf{e}_k^T \mathbf{y})$ and renormalizing has the effect of purging undesirable components without requiring any additional matrix vector products with A_σ .

9. The residual error of the purged vector $\mathbf{x}$ with respect to the original problem is

$$\|A\mathbf{x} - M\mathbf{x}\lambda\|_2 = \|M\mathbf{f}\|_2 \frac{|\mathbf{e}_k^T \mathbf{y}|}{|\theta|^2},$$

where $\lambda = \sigma + 1/\theta$. Since $|\theta|$ is usually quite large under the spectral transformation, this new residual is generally considerably smaller than the original.

References

- [BCR96] J. Baglama, D. Calvetti, and L. Reichel, Iterative methods for the computation of a few eigenvalues of a large symmetric matrix, *BIT*, 36 (3), 400–440 (1996).
- [BER04] C.A. Beattie, M. Embree, and J. Rossi, Convergence of restarted Krylov subspaces to invariant subspaces, *SIAM J. Matrix Anal. Appl.*, 25, 1074–1109 (2004).
- [BES05] C.A. Beattie, M. Embree, and D.C. Sorensen, Convergence of polynomial restart Krylov methods for eigenvalue computation, *SIAM Review*, 47 (3), 492–515 (2005).

- [DGK76] J. Daniel, W.B. Gragg, L. Kaufman, and G.W. Stewart, Reorthogonalization and stable algorithms for updating the Gram–Schmidt QR factorization, *Math. Comp.*, 30, 772–795 (1976).
- [DDH88] J.J. Dongarra, J. DuCroz, S. Hammarling, and R. Hanson, An extended set of Fortran basic linear algebra subprograms, *ACM Trans. Math. Softw.*, 14, 1–17 (1988).
- [ER80] T. Ericsson and A. Ruhe, The spectral transformation Lanczos method for the numerical solution of large sparse generalized symmetric eigenvalue problems, *Math. Comp.*, 35, (152), 1251–1268 (1980).
- [FR89] B. Fischer and L. Reichel, Newton interpolation in Fejér and Chebyshev points, *Math. Comp.*, 53, 265–278 (1989).
- [Gai87] D. Gaier, *Lectures on Complex Approximation*, Birkhäuser, Boston (1987).
- [GLS94] R.G. Grimes, J.G. Lewis, and H.D. Simon, A shifted block Lanczos algorithm for solving sparse symmetric generalized eigenproblems, *SIAM J. Matrix Anal. Appl.*, 15 (1), 228–272 (1994).
- [LSY98] R. Lehoucq, D.C. Sorensen, and C. Yang, *ARPACK Users Guide: Solution of Large Scale Eigenvalue Problems with Implicitly Restarted Arnoldi Methods*, SIAM Publications, Philadelphia (1998). (Software available at: <http://www.caam.rice.edu/software/ARPACK>.)
- [MS97] K. Meerbergen and A. Spence, Implicitly restarted Arnoldi with purification for the shift–invert transformation, *Math. Comp.*, 218, 667–689 (1997).
- [Mor96] R.B. Morgan, On restarting the Arnoldi method for large nonsymmetric eigenvalue problems, *Math. Comp.*, 65, 1213–1230 (1996).
- [Par80] B.N. Parlett, *The Symmetric Eigenvalue Problem*, Prentice-Hall, Upper Saddle River, NJ (1980).
- [Saa92] Y. Saad, *Numerical Methods for Large Eigenvalue Problems*, Manchester University Press, Manchester, U.K. (1992).
- [Sor92] D.C. Sorensen, Implicit application of polynomial filters in a k -step Arnoldi method, *SIAM J. Matrix Anal. Applic.*, 13: 357–385 (1992).
- [Sor02] D.C. Sorensen, Numerical methods for large eigenvalue problems, *Acta Numerica*, 11, 519–584 (2002).

45

Computation of the Singular Value Decomposition

Alan Kaylor Cline

The University of Texas at Austin

Inderjit S. Dhillon

The University of Texas at Austin

45.1	Singular Value Decomposition	45-1
45.2	Algorithms for the Singular Value Decomposition ...	45-4
	References	45-12

45.1 Singular Value Decomposition

Definitions:

Given a complex matrix A having m rows and n columns, if σ is a nonnegative scalar and $\mathbf{u}$ and $\mathbf{v}$ are nonzero m - and n -vectors, respectively, such that

$$A\mathbf{v} = \sigma\mathbf{u} \quad \text{and} \quad A^*\mathbf{u} = \sigma\mathbf{v},$$

then σ is a **singular value** of A and $\mathbf{u}$ and $\mathbf{v}$ are corresponding **left** and **right singular vectors**, respectively. (For generality it is assumed that the matrices here are complex, although given these results, the analogs for real matrices are obvious.)

If, for a given positive singular value, there are exactly t linearly independent corresponding right singular vectors and t linearly independent corresponding left singular vectors, the singular value has **multiplicity** t and the space spanned by the right (left) singular vectors is the corresponding **right (left) singular space**.

Given a complex matrix A having m rows and n columns, the matrix product $U\Sigma V^*$ is a **singular value decomposition** for a given matrix A if

- U and V , respectively, have orthonormal columns.
- Σ has nonnegative elements on its principal diagonal and zeros elsewhere.
- $A = U\Sigma V^*$.

Let p and q be the number of rows and columns of Σ . U is $m \times p$, $p \leq m$, and V is $n \times q$ with $q \leq n$.

There are three **standard forms** of the SVD. All have the i th diagonal value of Σ denoted σ_i and ordered as follows: $\sigma_1 \geq \sigma_2 \geq \dots \geq \sigma_k$, and r is the index such that $\sigma_r > 0$ and either $k = r$ or $\sigma_{r+1} = 0$.

1. $p = m$ and $q = n$. The matrix Σ is $m \times n$ and has the same dimensions as A (see Figures 45.1 and 45.2).
2. $p = q = \min\{m, n\}$. The matrix Σ is square (see Figures 45.3 and 45.4).
3. If $p = q = r$, the matrix Σ is square. This form is called a **reduced SVD** and denoted is by $\hat{U}\hat{\Sigma}\hat{V}^*$ (see Figures 45.5 and 45.6).

$$\boxed{A} = \boxed{U} \boxed{\Sigma} \boxed{V^*}$$

FIGURE 45.1 The first form of the singular value decomposition where $m \geq n$.

$$\boxed{A} = \boxed{U} \boxed{\Sigma} \boxed{V^*}$$

FIGURE 45.2 The first form of the singular value decomposition where $m < n$.

$$\boxed{A} = \boxed{U} \boxed{\Sigma} \boxed{V^*}$$

FIGURE 45.3 The second form of the singular value decomposition where $m \geq n$.

$$\boxed{A} = \boxed{U} \boxed{\Sigma} \boxed{V^*}$$

FIGURE 45.4 The second form of the singular value decomposition where $m < n$.

$$\boxed{A} = \boxed{\hat{U}} \boxed{\hat{\Sigma}} \boxed{\hat{V}^*}$$

FIGURE 45.5 The third form of the singular value decomposition where $r \leq n \leq m$.

$$\boxed{A} = \boxed{\hat{U}} \boxed{\hat{\Sigma}} \boxed{\hat{V}^*}$$

FIGURE 45.6 The third form of the singular value decomposition where $r \leq m < n$.

Facts:

The results can be found in [GV96, pp. 70–79]. Additionally, see Chapter 5.6 for introductory material and examples of SVDs, Chapter 17 for additional information on singular value decomposition, Chapter 15 for information on perturbations of singular values and vectors, and Section 39.9 for information about numerical rank.

1. If $U\Sigma V^*$ is a singular value decomposition for a given matrix A , then the diagonal elements $\{\sigma_i\}$ of Σ are singular values of A . The columns $\{\mathbf{u}_i\}_{i=1}^p$ of U and $\{\mathbf{v}_i\}_{i=1}^q$ of V are left and right singular vectors of A , respectively.
2. If $m \geq n$, the first standard form of the SVD can be found as follows:
 - (a) Let $A^*A = V\Lambda V^*$ be an eigenvalue decomposition for the Hermitian, positive semidefinite $n \times n$ matrix A^*A such that Λ is diagonal (with the diagonal entries in nonincreasing order) and V is unitary.
 - (b) Let the $m \times n$ matrix Σ have zero off-diagonal elements and for $i = 1, \dots, n$ let σ_i , the i th diagonal element of Σ , equal $\sqrt{\lambda_i}$, the positive square root of the i th diagonal element of Λ .
 - (c) For $i = 1, \dots, n$, let the $m \times m$ matrix U have i th column, $\mathbf{u}_i$, equal to $1/\sigma_i A\mathbf{v}_i$ if $\sigma_i \neq 0$. If $\sigma_i = 0$, let $\mathbf{u}_i$ be of unit length and orthogonal to all $\mathbf{u}_j$ for $j \neq i$, then $U\Sigma V^*$ is a singular decomposition of A .
3. If $m < n$ the matrix A^* has a singular value decomposition $U\Sigma V^*$ and $V\Sigma^T U^*$ is a singular value decomposition for A . The diagonal elements of Σ are the square roots of the eigenvalues of AA^* . The eigenvalues of A^*A are those of AA^* plus $n - m$ zeros. The notation Σ^T rather than Σ^* is used because in this case the two are identical and the transpose is more suggestive. All elements of Σ are real so that taking complex conjugates has no effect.
4. The value of r , the number of nonzero singular values, is the rank of A .
5. If A is real, then U and V (in addition to Σ) can be chosen real in any of the forms of the SVD.
6. The range of A is exactly the subspace of $\mathbb{C}^m$ spanned by the r columns of U that correspond to the positive singular values.
7. In the first form, the null space of A is that subspace of $\mathbb{C}^n$ spanned by the $n - r$ columns of V that correspond to zero singular values.
8. In reducing from the first form to the third (reduced) form, a basis for the null space of A has been discarded if columns of V have been deleted. A basis for the space orthogonal to the range of A (i.e., the null space of A^*) has been discarded if columns of U have been deleted.
9. In the first standard form of the SVD, U and V are unitary.
10. The second form can be obtained from the first form simply by deleting columns $n + 1, \dots, m$ of U and the corresponding rows of S , if $m > n$, or by deleting columns $m + 1, \dots, n$ of V and the corresponding columns of S , if $m < n$. If $m \neq n$, then only one of U and V is square and either $UU^* = I_m$ or $VV^* = I_n$ fails to hold. Both $U^*U = I_p$ and $V^*V = I_p$.
11. The reduced (third) form can be obtained from the second form by taking only the $r \times r$ principle submatrix of Σ , and only the first r columns of U and V . If A is rank deficient (i.e., $r < \min\{m, n\}$), then neither U nor V is square and neither U^*U nor V^*V is an identity matrix.
12. If $p < m$, let $\tilde{U}$ be an $m \times (m - p)$ matrix of columns that are mutually orthonormal to one another as well as to the columns of U and define the $m \times m$ unitary matrix

$$\widehat{U} = [U \quad \tilde{U}].$$

If $q < n$, let $\tilde{V}$ be an $n \times (n - q)$ matrix of columns that are mutually orthonormal to one another as well as to the columns of V and define the $n \times n$ unitary matrix

$$\widehat{V} = [V \quad \tilde{V}].$$

Let $\widehat{\Sigma}$ be the $m \times n$ matrix

$$\widehat{\Sigma} = \begin{bmatrix} \Sigma & 0 \\ 0 & 0 \end{bmatrix}.$$

Then

$$A = \widehat{U} \widehat{\Sigma} \widehat{V}^*, \quad A\widehat{V} = \widehat{U} \widehat{\Sigma}^*, \quad A^* = \widehat{V} \widehat{\Sigma}^T \widehat{U}^*, \quad \text{and} \quad A^* \widehat{U} = \widehat{V} \widehat{\Sigma}^T.$$

13. Let $U \Sigma V^*$ be a singular value decomposition for A , an $m \times n$ matrix of rank r . Then:
 - (a) There are exactly r positive elements of Σ and they are the square roots of the r positive eigenvalues of A^*A (and also AA^*) with the corresponding multiplicities.
 - (b) The columns of V are eigenvectors of A^*A ; more precisely, $\mathbf{v}_j$ is a normalized eigenvector of A^*A corresponding to the eigenvalue σ_j^2 , and $\mathbf{u}_j$ satisfies $\sigma_j \mathbf{u}_j = A\mathbf{v}_j$.
 - (c) Alternatively, the columns of U are eigenvectors of AA^* ; more precisely, $\mathbf{u}_j$ is a normalized eigenvector of AA^* corresponding to the eigenvalue σ_j^2 , and $\mathbf{v}_j$ satisfies $\sigma_j \mathbf{v}_j = A^* \mathbf{u}_j$.
14. The singular value decomposition $U \Sigma V^*$ is not unique. If $U \Sigma V^*$ is a singular value decomposition, so is $(-U) \Sigma (-V^*)$. The singular values may be arranged in any order if the columns of singular vectors in U and V are reordered correspondingly.
15. If the singular values are in nonincreasing order then the only option for the construction of Σ is the choice for its dimensions p and q and these must satisfy $r \leq p \leq m$ and $r \leq q \leq n$.
16. If A is square and if the singular values are ordered in a nonincreasing fashion, the matrix Σ is unique.
17. Corresponding to a simple (i.e., nonrepeated) singular value σ_j , the left and right singular vectors, $\mathbf{u}_j$ and $\mathbf{v}_j$, are unique up to scalar multiples of modulus one. That is, if $\mathbf{u}_j$ and $\mathbf{v}_j$ are singular vectors, then for any real value of θ so are $e^{i\theta} \mathbf{u}_j$ and $e^{i\theta} \mathbf{v}_j$, but no other vectors are singular vectors corresponding to σ_j .
18. Corresponding to a repeated singular value, the associated left singular vectors $\mathbf{u}_j$ and right singular vectors $\mathbf{v}_j$ may be selected in any fashion such that they span the proper subspace. Thus, if $\mathbf{u}_{j_1}, \dots, \mathbf{u}_{j_r}$ and $\mathbf{v}_{j_1}, \dots, \mathbf{v}_{j_r}$ are the left and right singular vectors corresponding to a singular value σ_j of multiplicity s , then so are $\mathbf{u}'_{j_1}, \dots, \mathbf{u}'_{j_r}$ and $\mathbf{v}'_{j_1}, \dots, \mathbf{v}'_{j_r}$ if and only if there exists an $s \times s$ unitary matrix Q such that $[\mathbf{u}'_{j_1}, \dots, \mathbf{u}'_{j_r}] = [\mathbf{u}_{j_1}, \dots, \mathbf{u}_{j_r}] Q$ and $[\mathbf{v}'_{j_1}, \dots, \mathbf{v}'_{j_r}] = [\mathbf{v}_{j_1}, \dots, \mathbf{v}_{j_r}] Q$.

Examples:

For examples illustrating SVD see Section 5.6.

45.2 Algorithms for the Singular Value Decomposition

Generally, algorithms for computing singular values are analogs of algorithms for computing eigenvalues of symmetric matrices. See Chapter 42 and Chapter 46 for additional information. The idea is always to find square roots of eigenvalues of $A^T A$ without actually computing $A^T A$. As before, we assume the matrix A whose singular values or singular vectors we seek is $m \times n$. All algorithms assume $m \geq n$; if $m < n$, the algorithms may be applied to A^T . To avoid undue complication, all algorithms will be presented as if the matrix is real. Nevertheless, each algorithm has an extension for complex matrices. Algorithm 1 is presented in three parts. It is analogous to the QR algorithm for symmetric matrices. The developments for it can be found in [GK65], [GK68], [BG69], and [GR70]. Algorithm 1a is a Householder reduction of a matrix to bidiagonal form. Algorithm 1c is a step to be used iteratively in Algorithm 1b. Algorithm 2 computes the singular values and singular vectors of a bidiagonal matrix to high relative accuracy [DK90], [Dem97].

Algorithm 3 gives a “Squareroot-free” method to compute the singular values of a bidiagonal matrix to high relative accuracy — it is the method of choice when only singular values are desired [Rut54], [Rut90], [FP94], [PM00]. Algorithm 4 computes the singular values of an $n \times n$ bidiagonal matrix by the bisection method, which allows k singular values to be computed in $O(kn)$ time. By specifying the input tolerance tol appropriately, Algorithm 4 can also compute the singular values to high relative accuracy. Algorithm 5 computes the SVD of a bidiagonal by the divide and conquer method [GE95]. The most recent method, based on the method of multiple relatively robust representations (not presented here), is the fastest and allows computation of k singular values as well as the corresponding singular vectors of a bidiagonal matrix in $O(kn)$ time [DP04a], [DP04b], [GL03], [WLV05]. All of the above mentioned methods first reduce the matrix to bidiagonal form. The following algorithms iterate directly on the input matrix. Algorithms 6 and 7 are analogous to the Jacobi method for symmetric matrices. Algorithm 6 — also known as the “one-sided Jacobi method for SVD” — can be found in [Hes58] and Algorithm 7 can be found in [Kog55] and [FH60]. Algorithm 7 begins with an orthogonal reduction of the $m \times n$ input matrix so that all the nonzeros lie in the upper $n \times n$ portion. (Although this algorithm was named biorthogonalization in [FH60], it is not the biorthogonalization found in certain iterative methods for solving linear equations.) Many of the algorithms require a tolerance ε to control termination. It is suggested that ε be set to a small multiple of the unit round off precision ε_o .

Algorithm 1a: Householder reduction to bidiagonal form:

Input: m, n, A where A is $m \times n$.

Output: B, U, V so that B is upper bidiagonal, U and V are products of Householder matrices, and $A = UB V^T$.

1. $B \leftarrow A$. (This step can be omitted if A is to be overwritten with B .)
2. $U = I_{m \times n}$.
3. $V = I_{n \times n}$.
4. For $k = 1, \dots, n$
 - a. Determine Householder matrix Q_k with the property that:
 - Left multiplication by Q_k leaves components $1, \dots, k-1$ unaltered, and
$$Q_k \begin{bmatrix} 0 \\ \vdots \\ 0 \\ b_{k-1,k} \\ b_{k,k} \\ b_{k+1,k} \\ \vdots \\ b_{m,k} \end{bmatrix} = \begin{bmatrix} 0 \\ \vdots \\ 0 \\ b_{k-1,k} \\ s \\ 0 \\ \vdots \\ 0 \end{bmatrix}, \text{ where } s = \pm \sqrt{\sum_{i=k}^m b_{i,k}^2}.$$
 - b. $B \leftarrow Q_k B$.
 - c. $U \leftarrow U Q_k$.
 - d. If $k \leq n-2$, determine Householder matrix P_{k+1} with the property that:
 - Right multiplication by P_{k+1} leaves components $1, \dots, k$ unaltered, and
 - $[0 \cdots 0 \quad b_{k,k} \quad b_{k,k+1} \quad b_{k,k+2} \cdots b_{k,n}] P_{k+1} = [0 \cdots 0 \quad b_{k,k} \quad s \quad 0 \cdots 0]$,
where $s = \pm \sqrt{\sum_{j=k+1}^n b_{k,j}^2}$.
 - e. $B \leftarrow B P_{k+1}$.
 - f. $V \leftarrow P_{k+1} V$.

Algorithm 1b: Golub–Reinsch SVD:

Input: m, n, A where A is $m \times n$.

Output: Σ, U, V so that Σ is diagonal, U and V have orthonormal columns, U is $m \times n$, V is $n \times n$, and $A = U \Sigma V^T$.

1. Apply Algorithm 1a to obtain B, U, V so that B is upper bidiagonal, U and V are products of Householder matrices, and $A = U B V^T$.
2. Repeat:
 - a. If for any $i = 1, \dots, n-1$, $|b_{i,i+1}| \leq \varepsilon (|b_{i,i}| + |b_{i+1,i+1}|)$, set $b_{i,i+1} = 0$.
 - b. Determine the smallest p and the largest q so that B can be blocked as

$$B = \begin{bmatrix} B_{1,1} & 0 & 0 \\ 0 & B_{2,2} & 0 \\ 0 & 0 & B_{3,3} \end{bmatrix} \quad \begin{matrix} p \\ n-p-q \\ q \end{matrix}$$

where $B_{3,3}$ is diagonal and $B_{2,2}$ has no zero superdiagonal entry.

- c. If $q = n$, set Σ = the diagonal portion of B **STOP**.
- d. If for $i = p+1, \dots, n-q-1$, $b_{i,i} = 0$, then

Apply Givens rotations so that $b_{i,i+1} = 0$ and $B_{2,2}$ is still upper bidiagonal. (For details, see [GL96, p. 454].)

else

Apply Algorithm 1c to n, B, U, V, p, q .

Algorithm 1c: Golub–Kahan SVD step:

Input: n, B, Q, P, p, q where B is $n \times n$ and upper bidiagonal, Q and P have orthogonal columns, and $A = Q B P^T$.

Output: B, Q, P so that B is upper bidiagonal, $A = Q B P^T$, Q and P have orthogonal columns, and the output B has smaller off-diagonal elements than the input B . In storage, B, Q , and P are overwritten.

1. Let $B_{2,2}$ be the diagonal block of B with row and column indices $p+1, \dots, n-q$.
2. Set C = lower, right 2×2 submatrix of $B_{2,2}^T B_{2,2}$.
3. Obtain eigenvalues λ_1, λ_2 of C . Set μ = whichever of λ_1, λ_2 that is closer to $c_{2,2}$.
4. $k = p+1, \alpha = b_{k,k}^2 - \mu, \beta = b_{k,k} b_{k,k+1}$.
5. For $k = p+1, \dots, n-q-1$
 - a. Determine $c = \cos(\theta)$ and $s = \sin(\theta)$ with the property that:

$$\begin{bmatrix} \alpha & \beta \end{bmatrix} \begin{bmatrix} c & s \\ -s & c \end{bmatrix} = \begin{bmatrix} \sqrt{\alpha^2 + \beta^2} & 0 \end{bmatrix}.$$

- b. $B \leftarrow B R_{k,k+1}(c, s)$ where $R_{k,k+1}(c, s)$ is the Givens rotation matrix that acts on columns k and $k+1$ during right multiplication.
- c. $P \leftarrow P R_{k,k+1}(c, s)$.
- d. $\alpha = b_{k,k}, \beta = b_{k+1,k}$.
- e. Determine $c = \cos(\theta)$ and $s = \sin(\theta)$ with the property that:

$$\begin{bmatrix} c & -s \\ s & c \end{bmatrix} \begin{bmatrix} \alpha \\ \beta \end{bmatrix} = \begin{bmatrix} \sqrt{\alpha^2 + \beta^2} \\ 0 \end{bmatrix}.$$

- f. $B \leftarrow R_{k,k+1}(c, -s) B$, where $R_{k,k+1}(c, -s)$ is the Givens rotation matrix that acts on rows k and $k+1$ during left multiplication.
- g. $Q \leftarrow Q R_{k,k+1}(c, s)$.
- h. if $k \leq n-q-1$ $\alpha = b_{k,k+1}, \beta = b_{k,k+2}$.

Algorithm 2a: High Relative Accuracy Bidiagonal SVD:

Input: n, B where B is an $n \times n$ upper bidiagonal matrix.

Output: Σ is an $n \times n$ diagonal matrix, U and V are orthogonal $n \times n$ matrices, and $B = U\Sigma V^T$.

1. Compute $\underline{\sigma}$ to be a reliable underestimate of $\sigma_{\min}(B)$ (for details, see [DK90]).
2. Compute $\overline{\sigma} = \max_i(b_{i,i}, b_{i,i+1})$.
3. Repeat:
 - a. For all $i = 1, \dots, n-1$, set $b_{i,i+1} = 0$ if a relative convergence criterion is met (see [DK90] for details).
 - b. Determine the smallest p and largest q so that B can be blocked as

$$B = \begin{bmatrix} B_{1,1} & 0 & 0 \\ 0 & B_{2,2} & 0 \\ 0 & 0 & B_{3,3} \end{bmatrix} \begin{matrix} p \\ n-p-q \\ q \end{matrix}$$

where $B_{3,3}$ is diagonal and $B_{2,2}$ has no zero superdiagonal entry.

- c. If $q = n$, set $\Sigma =$ the diagonal portion of B . **STOP**.
- d. If for $i = p+1, \dots, n-q-1$, $b_{i,i} = 0$, then

Apply Givens rotations so that $b_{i,i+1} = 0$ and $B_{2,2}$ is still upper bidiagonal. (For details, see [GV96, p. 454].)

else

Apply Algorithm 2b with $n, B, U, V, p, q, \overline{\sigma}, \underline{\sigma}$ as inputs.

Algorithm 2b: Demmel–Kahan SVD step:

Input: $n, B, Q, P, p, q, \overline{\sigma}, \underline{\sigma}$ where B is $n \times n$ and upper bidiagonal, Q and P have orthogonal columns such that $A = QB P^T$, $\overline{\sigma} \approx \|B\|$ and $\underline{\sigma}$ is an underestimate of $\sigma_{\min}(B)$.

Output: B, Q, P so that B is upper bidiagonal, $A = QB P^T$, Q and P have orthogonal columns, and the output B has smaller off-diagonal elements than the input B . In storage, B, Q , and P are overwritten.

1. Let $B_{2,2}$ be the diagonal block of B with row and column indices $p+1, \dots, n-q$.
2. If $\text{tol}^* \underline{\sigma} \leq \varepsilon_0 \overline{\sigma}$, then
 - a. $c' = c = 1$.
 - b. For $k = p+1, n-q-1$
 - $\alpha = cb_{k,k}; \beta = b_{k,k+1}$.
 - Determine c and s with the property that:

$$\begin{bmatrix} \alpha & \beta \end{bmatrix} \begin{bmatrix} c & s \\ -s & c \end{bmatrix} = \begin{bmatrix} r & 0 \end{bmatrix}, \text{ where } r = \sqrt{\alpha^2 + \beta^2}.$$

- If $k \neq p+1$, $b_{k-1,k} = s'r$.
- $P \leftarrow P R_{k,k+1}(c, s)$, where $R_{k,k+1}(c, s)$ is the Givens rotation matrix that acts on columns k and $k+1$ during right multiplication.
- $\alpha = c'r, \beta = sb_{k+1,k+1}$.

(continued)

Algorithm 2b: Demmel–Kahan SVD step: (Continued)

- Determine c' and s' with the property that:

$$\begin{bmatrix} c' & -s' \\ s' & c' \end{bmatrix} \begin{bmatrix} \alpha \\ \beta \end{bmatrix} = \begin{bmatrix} \sqrt{\alpha^2 + \beta^2} \\ 0 \end{bmatrix}.$$

- $Q \leftarrow QR_{k,k+1}(c, -s)$, where $R_{k,k+1}(c, -s)$ is the Givens rotation matrix that acts on rows k and $k + 1$ during left multiplication.
 - $b_{k,k} = \sqrt{\alpha^2 + \beta^2}$.
- c. $b_{n-q-1,n-q} = (b_{n-q,n-q}c)s'$; $b_{n-q,n-q} = (b_{n-q,n-q}c)c'$.
Else
- d. Apply Algorithm 1c to n, B, Q, P, p, q .

Algorithm 3a: High Relative Accuracy Bidiagonal Singular Values:

Input: n, B where B is an $n \times n$ upper bidiagonal matrix.

Output: Σ is an $n \times n$ diagonal matrix containing the singular values of B .

1. Square the diagonal and off-diagonal elements of B to form the arrays $\mathbf{s}$ and $\mathbf{e}$, respectively, i.e., for $i = 1, \dots, n-1$, $s_i = b_{i,i}^2$, $e_i = b_{i,i+1}^2$, end for $s_n = b_{n,n}^2$.
2. Repeat:
 - a. For all $i = 1, \dots, n-1$, set $e_i = 0$ if a relative convergence criterion is met (see [PM00] for details).
 - b. Determine the smallest p and largest q so that B can be blocked as

$$B = \begin{bmatrix} B_{1,1} & 0 & 0 \\ 0 & B_{2,2} & 0 \\ 0 & 0 & B_{3,3} \end{bmatrix} \quad \begin{matrix} p \\ n-p-q \\ q \end{matrix}$$

where $B_{3,3}$ is diagonal and $B_{2,2}$ has no zero superdiagonal entry.

- c. If $q = n$, set $\Sigma = \sqrt{\text{diag}(\mathbf{s})}$. **STOP**.
- d. If for $i = p+1, \dots, n-q-1$, $s_i = 0$ then
Apply Givens rotations so that $e_i = 0$ and $B_{2,2}$ is still upper bidiagonal. (For details, see [GV96, p. 454].)
else
Apply Algorithm 3b with inputs $n, \mathbf{s}, \mathbf{e}$.

Algorithm 3b: Differential quotient-difference (dqds) step:

Input: $n, \mathbf{s}, \mathbf{e}$ where $\mathbf{s}$ and $\mathbf{e}$ are the squares of the diagonal and superdiagonal entries, respectively, of an $n \times n$ upper bidiagonal matrix.

Output: $\mathbf{s}$ and $\mathbf{e}$ are overwritten on output.

1. Choose μ by using a suitable shift strategy. The shift μ should be smaller than $\sigma_{\min}(B)^2$. See [FP94, PM00] for details.
2. $d = s_1 - \mu$.

Algorithm 3b: Differential quotient-difference (dqds) step: (Continued)

1. For $k = 1, \dots, n - 1$
 - a. $s_k = d + e_k$.
 - b. $t = s_{k+1}/s_k$.
 - c. $e_k = e_k t$.
 - d. $d = dt - \mu$.
 - e. If $d < 0$, go to step 1.
2. $s_n = d$.

Algorithm 4a: Bidiagonal Singular Values by Bisection:

Input: n, B, α, β, tol where $n \times n$ is a bidiagonal matrix, $[\alpha, \beta]$ is the input interval, and tol is the tolerance for the desired accuracy of the singular values.

Output: w is the output array containing the singular values of B that lie in $[\alpha, \beta]$.

1. $n_\alpha = \text{Negcount}(n, B, \alpha)$.
2. $n_\beta = \text{Negcount}(n, B, \beta)$.
3. If $n_\alpha = n_\beta$, there are no singular values in $[\alpha, \beta]$. **STOP**.
4. Put $[\alpha, n_\alpha, \beta, n_\beta]$ onto *Worklist*.
5. While *Worklist* is not empty do
 - a. Remove $[low, n_{low}, up, n_{up}]$ from *Worklist*.
 - b. $mid = (low + up)/2$.
 - c. If $(up - low < tol)$, then
 - For $i = n_{low} + 1, n_{up}, w(i - n_a) = mid$;
 Else
 - $n_{mid} = \text{Negcount}(n, B, mid)$.
 - If $n_{mid} > n_{low}$ then
Put $[low, n_{low}, mid, n_{mid}]$ onto *Worklist*.
 - If $n_{up} > n_{mid}$ then
Put $[mid, n_{mid}, up, n_{up}]$ onto *Worklist*.

Algorithm 4b: $\text{Negcount}(n, B, \mu)$:

Input: The $n \times n$ bidiagonal matrix B and a number μ .

Output: Negcount , i.e., the number of singular values smaller than μ is returned.

1. $t = -\mu$.
2. For $k = 1, \dots, n - 1$
 - $d = b_{k,k}^2 + t$.
 - If $(d < 0)$ then $\text{Negcount} = \text{Negcount} + 1$
 - $t = t * (b_{k,k+1}^2/d) - \mu$.
- End for
3. $d = b_{n,n}^2 + t$.
4. If $(d < 0)$, then $\text{Negcount} = \text{Negcount} + 1$.

Algorithm 5: DC_SVD(n, B, Σ, U, V): Divide and Conquer Bidiagonal SVD:

Input: n, B where B is an $(n+1) \times n$ lower bidiagonal matrix.

Output: Σ is an $n \times n$ diagonal matrix, U is an $(n+1) \times (n+1)$ orthogonal matrix, V is an orthogonal $n \times n$ matrix, so that $B = U\Sigma V^T$.

1. If $n < n_0$, then apply Algorithm 1b with inputs $n+1, n, B$ to get outputs Σ, U, V .
Else

$$\text{Let } B = \begin{pmatrix} B_1 & \alpha_k \mathbf{e}_k & 0 \\ 0 & \beta_k \mathbf{e}_1 & B_2 \end{pmatrix}, \text{ where } k = n/2.$$

- a. Call DC_SVD($k-1, B_1, \Sigma_1, U_1, W_1$).
- b. Call DC_SVD($n-k, B_2, \Sigma_2, U_2, W_2$).
- c. Partition $U_i = (Q_i \quad \mathbf{q}_i)$, for $i = 1, 2$, where $\mathbf{q}_i$ is a column vector.
- d. Extract $l_1 = Q_1^T \mathbf{e}_k, \lambda_1 = \mathbf{q}_1^T \mathbf{e}_k, l_2 = Q_2^T \mathbf{e}_1, \lambda_2 = \mathbf{q}_2^T \mathbf{e}_1$.
- e. Partition B as

$$B = \begin{pmatrix} c_0 \mathbf{q}_1 & Q_1 & 0 & -s_0 \mathbf{q}_1 \\ s_0 \mathbf{q}_2 & 0 & Q_2 & c_0 \mathbf{q}_2 \end{pmatrix} \begin{pmatrix} r_0 & 0 & 0 \\ \alpha_k l_1 & \Sigma_1 & 0 \\ \beta_k l_2 & 0 & \Sigma_2 \\ 0 & 0 & 0 \end{pmatrix} \begin{pmatrix} 0 & W_1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & W_2 \end{pmatrix}^T = (Q \quad q) \begin{pmatrix} M \\ 0 \end{pmatrix} W^T$$

where $r_0 = \sqrt{(\alpha_k \lambda_1)^2 + (\beta_k \lambda_2)^2}, c_0 = \alpha_k \lambda_1 / r_0, s_0 = \beta_k \lambda_2 / r_0$.

- f. Compute the singular values of M by solving the secular equation

$$f(w) = 1 + \sum_{k=1}^n \frac{z_k^2}{d_k^2 - w^2} = 0,$$

and denote the computed singular values by $\hat{w}_1, \hat{w}_2, \dots, \hat{w}_n$.

- g. For $i = 1, \dots, n$, compute

$$\hat{z}_i = \sqrt{(\hat{w}_n^2 - d_i^2) \prod_{k=1}^{i-1} \frac{(\hat{w}_k^2 - d_i^2)}{(d_k^2 - d_i^2)} \prod_{k=1}^{n-1} \frac{(\hat{w}_k^2 - d_i^2)}{(d_{k+1}^2 - d_i^2)}}.$$

- h. For $i = 1, \dots, n$, compute the singular vectors

$$\mathbf{u}_i = \left(\frac{\hat{z}_1}{d_1^2 - \hat{w}_i^2}, \dots, \frac{\hat{z}_n}{d_n^2 - \hat{w}_i^2} \right) / \sqrt{\sum_{k=1}^n \frac{\hat{z}_k^2}{(d_k^2 - \hat{w}_i^2)^2}},$$

$$\mathbf{v}_i = \left(-1, \frac{d_2 \hat{z}_2}{d_2^2 - \hat{w}_i^2}, \dots, \frac{d_n \hat{z}_n}{d_n^2 - \hat{w}_i^2} \right) / \sqrt{1 + \sum_{k=2}^n \frac{(d_k \hat{z}_k)^2}{(d_k^2 - \hat{w}_i^2)^2}}$$

and let $U = [\mathbf{u}_1, \dots, \mathbf{u}_n], V = [\mathbf{v}_1, \dots, \mathbf{v}_n]$.

- i. Return $\Sigma = \begin{pmatrix} \text{diag}(\hat{w}_1, \hat{w}_2, \dots, \hat{w}_n) \\ 0 \end{pmatrix}, U \leftarrow (QU \quad q), V \leftarrow WV$.

Algorithm 6: Biorthogonalization SVD:Input: m, n, A where A is $m \times n$.Output: Σ, U, V so that Σ is diagonal, U and V have orthonormal columns, U is $m \times n$, V is $n \times n$, and $A = U\Sigma V^T$.1. $U \leftarrow A$. (This step can be omitted if A is to be overwritten with U .)2. $V = I_{n \times n}$.3. Set $N^2 = \left(\sum_{i=1}^n \sum_{j=1}^n u_{i,j}^2 \right)$, $s = 0$, and $first = \mathbf{true}$.4. Repeat until $s^{1/2} \leq \varepsilon^2 N^2$ and $first = \mathbf{false}$.a. Set $s = 0$ and $first = \mathbf{false}$.b. For $i = 1, \dots, n-1$.i. For $j = i+1, \dots, n$

$$\bullet s \leftarrow s + \left(\sum_{k=1}^m u_{k,i} u_{k,j} \right)^2.$$

 $\bullet$ Determine $d_1, d_2, c = \cos(\theta)$, and $s = \sin(\varphi)$ such that:

$$\begin{bmatrix} c & -s \\ s & c \end{bmatrix} \begin{bmatrix} \sum_{k=1}^m u_{k,i}^2 & \sum_{k=1}^m u_{k,i} u_{k,j} \\ \sum_{k=1}^m u_{k,i} u_{k,j} & \sum_{k=1}^m u_{k,j}^2 \end{bmatrix} \begin{bmatrix} c & s \\ -s & c \end{bmatrix} = \begin{bmatrix} d_1 & 0 \\ 0 & d_2 \end{bmatrix}.$$

 $\bullet U \leftarrow U R_{i,j}(c, s)$ where $R_{i,j}(c, s)$ is the Givens rotation matrix that acts on columns i and j during right multiplication.

 $\bullet V \leftarrow V R_{i,j}(c, s)$.
5. For $i = 1, \dots, n$:

$$\text{a. } \sigma_i = \sqrt{\sum_{k=1}^m u_{k,i}^2}.$$

b. $U \leftarrow U \Sigma^{-1}$.**Algorithm 7:** Jacobi Rotation SVD:Input: m, n, A where A is $m \times n$.Output: Σ, U, V so that Σ is diagonal, U and V have orthonormal columns, U is $m \times n$, V is $n \times n$, and $A = U\Sigma V^T$.1. $B \leftarrow A$. (This step can be omitted if A is to be overwritten with B .)2. $U = I_{m \times n}$.3. $V = I_{n \times n}$.4. If $m > n$, compute the QR factorization of B using Householder matrices so that $B \leftarrow QA$, where B is upper triangular, and let $U \leftarrow UQ$. (See A6 for details.)5. Set $N^2 = \sum_{i=1}^n \sum_{j=1}^n b_{i,j}^2$, $s = 0$, and $first = \mathbf{true}$.6. Repeat until $s \leq \varepsilon^2 N^2$ and $first = \mathbf{false}$.a. Set $s = 0$ and $first = \mathbf{false}$.b. For $i = 1, \dots, n-1$

(continued)

Algorithm 7: Jacobi Rotation SVD: (Continued)

i. For $j = i + 1, \dots, n$:

- $s = s + b_{i,j}^2 + b_{j,i}^2$.
- Determine $d_1, d_2, c = \cos(\theta)$ and $s = \sin(\varphi)$ with the property that d_1 and d_2 are positive and

$$\begin{bmatrix} c & -s \\ s & c \end{bmatrix} \begin{bmatrix} b_{i,i} & b_{i,j} \\ b_{j,i} & b_{j,j} \end{bmatrix} \begin{bmatrix} \hat{c} & \hat{s} \\ -\hat{s} & \hat{c} \end{bmatrix} = \begin{bmatrix} d_1 & 0 \\ 0 & d_2 \end{bmatrix}.$$

- $B \leftarrow R_{i,j}(c, s) B R_{i,j}(\hat{c}, -\hat{s})$ where $R_{i,j}(c, s)$ is the Givens rotation matrix that acts on rows i and j during left multiplication and $R_{i,j}(\hat{c}, -\hat{s})$ is the Givens rotation matrix that acts on columns i and j during right multiplication.
- $U \leftarrow U R_{i,j}(c, s)$.
- $V \leftarrow V R_{i,j}(\hat{c}, \hat{s})$.

7. Set Σ to the diagonal portion of B .

References

- [BG69] P.A. Businger and G.H. Golub. Algorithm 358: singular value decomposition of a complex matrix, *Comm. Assoc. Comp. Mach.*, 12:564–565, 1969.
- [Dem97] J.W. Demmel. *Applied Numerical Linear Algebra*, SIAM, Philadelphia, 1997.
- [DK90] J.W. Demmel and W. Kahan. Accurate singular values of bidiagonal matrices, *SIAM J. Stat. Comp.*, 11:873–912, 1990.
- [DP04a] I.S. Dhillon and B.N. Parlett., Orthogonal eigenvectors and relative gaps, *SIAM J. Mat. Anal. Appl.*, 25:858–899, 2004.
- [DP04b] I.S. Dhillon and B.N. Parlett. Multiple representations to compute orthogonal eigenvectors of symmetric tridiagonal matrices, *Lin. Alg. Appl.*, 387:1–28, 2004.
- [FH60] G.E. Forsythe and P. Henrici. The cyclic Jacobi method for computing the principle values of a complex matrix, *Proc. Amer. Math. Soc.*, 94:1–23, 1960.
- [FP94] V. Fernando and B.N. Parlett. Accurate singular values and differential qd algorithms, *Numer. Math.*, 67:191–229, 1994.
- [GE95] M. Gu and S.C. Eisenstat. A divide-and-conquer algorithm for the bidiagonal SVD, *SIAM J. Mat. Anal. Appl.*, 16:79–92, 1995.
- [GK65] G.H. Golub and W. Kahan. Calculating the Singular Values and Pseudoinverse of a Matrix, *SIAM J. Numer. Anal.*, Ser. B, 2:205–224, 1965.
- [GK68] G.H. Golub and W. Kahan. Least squares, singular values, and matrix approximations, *Aplikace Matematiky*, 13:44–51 1968.
- [GL03] B. Grosser and B. Lang. An $O(n^2)$ algorithm for the bidiagonal SVD, *Lin. Alg. Appl.*, 358:45–70, 2003.
- [GR70] G.H. Golub and C. Reinsch. Singular value decomposition and least squares solutions, *Numer. Math.*, 14:403–420, 1970.
- [GV96] G.H. Golub and C.F. Van Loan. *Matrix Computations*, 3rd ed., The Johns Hopkins University Press, Baltimore, MD, 1996.
- [Hes58] M.R. Hestenes. Inversion of matrices by biorthogonalization and related results, *J. SIAM*, 6:51–90, 1958.
- [Kog55] E.G. Kogbetliantz. Solutions of linear equations by diagonalization of coefficient matrix, *Quart. Appl. Math.*, 13:123–132, 1955.

- [PM00] B.N. Parlett and O.A. Marques. An implementation of the dqds algorithm (positive case), *Lin. Alg. Appl.*, 309:217–259, 2000.
- [Rut54] H. Rutishauser. Der Quotienten-Differenzen-Algorithmus, *Z. Angew Math. Phys.*, 5:233–251, 1954.
- [Rut90] H. Rutishauser. *Lectures on Numerical Mathematics*, Birkhauser, Boston, 1990, (English translation of *Vorlesungen über numerische mathematik*, Birkhauser, Basel, 1996).
- [WLV05] P.R. Willems, B. Lang, and C. Voemel. Computing the bidiagonal SVD using multiple relatively robust representations, LAPACK Working Note 166, TR# UCB/CSD-05-1376, University of California, Berkeley, 2005.
- [Wil65] J.H. Wilkinson. *The Algebraic Eigenvalue Problem*, Clarendon Press, Oxford, U.K., 1965.

46

Computing Eigenvalues and Singular Values to High Relative Accuracy

46.1	Accurate SVD and One-Sided Jacobi SVD Algorithm	46-2
46.2	Preconditioned Jacobi SVD Algorithm	46-5
46.3	Accurate SVD from a Rank Revealing Decomposition: Structured Matrices	46-7
46.4	Positive Definite Matrices	46-10
46.5	Accurate Eigenvalues of Symmetric Indefinite Matrices	46-14
	References	46-17

Zlatko Drmač
University of Zagreb

To compute the eigenvalues and singular values to high relative accuracy means to have a guaranteed number of accurate digits in all computed approximate values. If $\tilde{\lambda}_i$ is the computed approximation of λ_i , then the desirable high relative accuracy means that $|\lambda_i - \tilde{\lambda}_i| \leq \varepsilon |\lambda_i|$, where $0 \leq \varepsilon \ll 1$ independent of the ratio $|\lambda_i| / \max_j |\lambda_j|$. This is not always possible. The proper course of action is to first determine classes of matrices and classes of perturbations under which the eigenvalues (singular values) undergo only small relative changes. This means that the development of highly accurate algorithms is determined by the framework established by the perturbation theory.

Input to the perturbation theory is a perturbation matrix whose size is usually measured in a matrix norm. This may not be always adequate in numerical solutions of real world problems. If numerical information stored in the matrix represents a physical system, then choosing different physical units can give differently scaled rows and columns of the matrix, but representing the same physical system. Different scalings may be present because matrix entries represent quantities of different physical nature. It is possible that the smallest matrix entries are determined as accurately as the biggest ones. Choosing the most appropriate scaling from the application's point of view can be difficult task. It is then desirable that simple change of reference units do not cause instabilities in numerical algorithms because changing the description (scaling the data matrix) does not change the underlying physical system. This issue is often overlooked or ignored in numerical computations and it can be the cause of incorrectly computed and misinterpreted results with serious consequences.

This chapter describes algorithms for computation of the singular values and the eigenvalues of symmetric matrices to high relative accuracy. Accurate computation of eigenvectors and singular vectors is not considered. Singular values are discussed in sections 46.1 to 46.3; algorithms for computation of singular values are also given in Chapter 45.2.

Algorithms for computing the eigenvalues of symmetric matrices are given in Chapter 42. Sections 45.4 to 45.5 discuss numerical issues concerning the accuracy of the computed approximate eigenvalues, divided into the cases positive definite and symmetric indefinite (a negative definite matrix A is handled by applying methods for positive definite matrices to $-A$).

For symmetric H (which may contain initial uncertainty), perturbation theory determines whether or not it is numerically feasible to compute even the smallest eigenvalues with high relative accuracy. State of the art perturbation theory, which is a necessary prerequisite for algorithmic development, is given in Chapter 15. The insights from the perturbation theory are then capitalized in the development of numerical algorithms capable of achieving optimal theoretical accuracy. Unlike in the case of standard accuracy, positive definite and indefinite matrices are analyzed separately.

All matrices are assumed to be over the real field $\mathbb{R}$. Additional relevant preparatory results can be found in Chapter 8 and Chapter 17.

46.1 Accurate SVD and One-Sided Jacobi SVD Algorithm

Numerical computation of the SVD inevitably means computation with errors. How many digits in the computed singular values are provably accurate is answered by perturbation theory adapted to a particular algorithm. For dense matrices with no additional structure, the Jacobi SVD algorithm is proven to be more accurate than any other method that first bidiagonalizes the matrix. The simplest form is the one-sided Jacobi SVD introduced by Hestenes [Hes58]. It represents an implicit form of the classical Jacobi algorithm [Jac46] for symmetric matrices, with properly adjusted stopping criterion. Basic properties of classical Jacobi algorithm are listed in Section 42.7. Detailed analysis and implementation details can be found in [DV92], [Drm94], [Mat96], and [Drm97].

Definitions:

Two vectors $\mathbf{x}, \mathbf{y} \in \mathbb{R}^m$ are **numerically orthogonal** if $|\mathbf{x}^T \mathbf{y}| \leq \varepsilon \|\mathbf{x}\|_2 \|\mathbf{y}\|_2$, where $\varepsilon \geq 0$ is at most round-off unit ϵ times a moderate function of m .

The square matrix $\tilde{U}$ is a **numerically orthogonal matrix** if each pair of its columns is numerically orthogonal.

A numerical algorithm that computes approximations $\tilde{U} \approx U$, $\tilde{V} \approx V$ and $\tilde{\Sigma} \approx \Sigma$ of the SVD $A = U \Sigma V^T$ of A is **backward stable** if $\tilde{U}$, $\tilde{V}$ are numerically orthogonal and $\tilde{U} \tilde{\Sigma} \tilde{V}^T = A + \delta A$, with **backward error** δA small compared to A .

Facts:

1. There is no loss of generality in considering real tall matrices, i.e., $m \geq n$. In case $m < n$, consider A^T with the SVD $A^T = V \Sigma^T U^T$.
2. Let $\tilde{U} \approx U$, $\tilde{V} \approx V$, $\tilde{\Sigma} \approx \Sigma$ be the approximations of the SVD of $A = U \Sigma V^T$, computed by a backward stable algorithm as $A + \delta A = \tilde{U} \tilde{\Sigma} \tilde{V}^T$. Since the orthogonality of $\tilde{U}$ and $\tilde{V}$ cannot be guaranteed, the product $\tilde{U} \tilde{\Sigma} \tilde{V}^T$ in general does not represent an SVD. However, there exist orthogonal $\hat{U}$ close to $\tilde{U}$ and an orthogonal $\hat{V}$, close to $\tilde{V}$, such that $(I + E_1)(A + \delta A)(I + E_2) = \hat{U} \hat{\Sigma} \hat{V}^T$, where E_1, E_2 , which represent departure from orthogonality of $\tilde{U}, \tilde{V}$, are small in norm.
3. Different algorithms produce differently structured δA . Consider the singular values $\sigma_1 \geq \dots \geq \sigma_n$ and $\tilde{\sigma}_1 \geq \dots \geq \tilde{\sigma}_n$ of A and $A + \delta A$, respectively.
 - If $\|\delta A\|_2 \leq \varepsilon \|A\|_2$ and without any additional structure of δA (δA small in norm), then the best error bound in the singular values is $\max_{1 \leq i \leq n} |\sigma_i - \tilde{\sigma}_i| \leq \|\delta A\|_2$, i.e., $\max_{1 \leq i \leq n} \frac{|\sigma_i - \tilde{\sigma}_i|}{\sigma_i} \leq \kappa_2(A) \varepsilon$.

- [DV92] Let, for all i , the i th column $\delta \mathbf{a}_i$ of δA satisfy $\|\delta \mathbf{a}_i\|_2 \leq \varepsilon \|\mathbf{a}_i\|_2$, where $\mathbf{a}_i$ is the i th column of A . (δA is column-wise small perturbation.) Then $A + \delta A = (B + \delta B)D$, where $A = BD$, $D = \text{diag}(\|\mathbf{a}_1\|_2, \dots, \|\mathbf{a}_n\|_2)$, and $\|\delta B\|_F \leq \sqrt{n}\varepsilon$. Let A have full column rank and let p be the rank of δA . Then $\max_{1 \leq i \leq n} \frac{|\sigma_i - \tilde{\sigma}_i|}{\sigma_i} \leq \|\delta B B^\dagger\|_2 \leq \sqrt{p}\varepsilon \|B^\dagger\|_2$.
 - [vdS69] $\|B^\dagger\|_2 \leq \kappa_2(B) \leq \sqrt{n} \min_{S=\text{diag}} \kappa_2(AS) \leq \sqrt{n}\kappa_2(A)$. This implies that a numerical algorithm with column-wise small backward error δA computes more accurate singular values than an algorithm with backward error that is only small in norm.
4. In the process of computation of the SVD of A , an algorithm can produce intermediate matrix with singular values highly sensitive even to smallest entry-wise rounding errors.
 5. The matrix A may have initial uncertainty on input and, in fact, $A = A_0 + \delta A_0$, where A_0 is the true unknown data matrix and δA_0 the initial error already present in A . If δA generated by the algorithm is comparable with δA_0 , the computed SVD is as accurate as warranted by the data.
 6. [Hes58] If $H = A^T A$, then the classical Jacobi algorithm can be applied to H implicitly. The one-sided (or implicit) Jacobi SVD method starts with general $m \times n$ matrix $A^{(0)} = A$ and it generates the sequence $A^{(k+1)} = A^{(k)} V^{(k)}$, where the matrix $V^{(k)}$ is the plane rotation as in the classical symmetric Jacobi method for the matrix $H^{(k)} = (A^{(k)})^T A^{(k)}$. Only $H^{(k)}[\{i_k, j_k\}]$ is needed to determine $V^{(k)}$, where (i_k, j_k) is the pivot pair determined by pivot strategy.
 7. In the one-sided Jacobi SVD algorithm applied to $A \in \mathbb{R}^{m \times n}$, $m \geq n$, $A^{(k)}$ tends to $U\Sigma$, with diagonal $\Sigma = \text{diag}(\sigma_1, \dots, \sigma_n)$ carrying the singular values. If A has full column rank, then the columns of U are the n corresponding left singular vectors. If $\text{rank}(A) < n$, then for each $\sigma_i = 0$ the i th column of U is zero. The m -rank(A) left singular vectors from the orthogonal complement of $\text{range}(A)$ cannot be computed using one-sided Jacobi SVD. The accumulated product $V^{(0)} V^{(1)} \dots$ converges to orthogonal matrix V of right singular vectors.
 8. Simple implementation of Jacobi rotation in the one-sided Jacobi SVD algorithm (one-sided Jacobi rotation) is given in Algorithm 1. At any moment in the algorithm, $d_1, \dots, d_n$ contain the squared Euclidean norms of the columns of current $A^{(k)}$, and ξ stores the Euclidean inner product of the pivot columns in the k th step.

Algorithm 1: One-sided Jacobi rotationROTATE ($A_{1:m,i}, A_{1:m,j}, d_i, d_j, \xi, [V_{1:m,i}, V_{1:m,j}]$)

- 1: $\vartheta = \frac{d_j - d_i}{2 \cdot \xi}; t = \frac{\text{sign}(\vartheta)}{|\vartheta| + \sqrt{1 + \vartheta^2}}; c = \frac{1}{\sqrt{1 + t^2}}; s = t \cdot c;$
- 2: $[A_{1:m,i} \ A_{1:m,j}] = [A_{1:m,i} \ A_{1:m,j}] \begin{bmatrix} c & s \\ -s & c \end{bmatrix};$
- 3: $d_i = d_i - t \cdot \xi; d_j = d_j + t \cdot \xi;$
- 4: **if** V is wanted, **then**
- 5: $[V_{1:m,i} \ V_{1:m,j}] = [V_{1:m,i} \ V_{1:m,j}] \begin{bmatrix} c & s \\ -s & c \end{bmatrix}$
- 6: **end if**

In case of cancellation in computation of the smaller of d_i, d_j , the value is refreshed by explicit computation of the squared norm of the corresponding column.

9. [dR89], [DV92] Numerical convergence of the one-sided Jacobi SVD algorithm (Algorithm 2) is declared at step k if $\max_{i \neq j} \frac{|(A_{1:m,i}^{(k)})^T A_{1:m,j}^{(k)}|}{\|A_{1:m,i}^{(k)}\|_2 \|A_{1:m,j}^{(k)}\|_2} \leq \zeta \approx m\varepsilon$. This stopping criterion guarantees that computed approximation $\tilde{A}^{(k)}$ of $A^{(k)}$ can be written as $\tilde{U}\tilde{\Sigma}$, where the columns of $\tilde{U}$ are numerically orthogonal, and $\tilde{\Sigma}$ is diagonal.

Algorithm 2: One-sided Jacobi SVD (de Rijk's row-cyclic pivoting)

```

( $U, \Sigma, [V]$ ) = SVD0( $A$ )
 $\zeta = m\epsilon$ ;  $\hat{p} = n(n-1)/2$ ;  $s = 0$ ; convergence = false;
if  $V$  is wanted, then initialize  $V = I_n$  end if
for  $i = 1$  to  $n$  do  $d_i = A_{1:m,i}^T A_{1:m,i}$  end for;
repeat
 $s = s + 1$ ;  $p = 0$ ;
for  $i = 1$  to  $n-1$  do
    find index  $i_0$  such that  $d_{i_0} = \max_{i \leq \ell \leq n} d_\ell$ ; swap( $d_i, d_{i_0}$ );
    swap( $A_{1:m,i}, A_{1:m,i_0}$ ); swap( $V_{1:m,i}, V_{1:m,i_0}$ );
    for  $j = i+1$  to  $n$  do
         $\xi = A_{1:m,i}^T A_{1:m,j}$ ;
        if  $|\xi| > \zeta \sqrt{d_i d_j}$  then % apply Jacobi rotation
            call ROTATE( $A_{1:m,i}, A_{1:m,j}, d_i, d_j, \xi, [V_{1:m,i}, V_{1:m,j}]$ );
        else  $p = p + 1$  end if
    end for
end for
if  $p = \hat{p}$ , then convergence = true; go to ► end if
until  $s > 30$ 
► if convergence, then % numerical orthogonality reached
    for  $i = 1$  to  $n$  do  $\Sigma_{ii} = \sqrt{d_i}$ ;  $U_{1:m,i} = A_{1:m,i} \Sigma_{ii}^{-1}$  end for
else
    Error: Numerical convergence did not occur after 30 sweeps.
end if

```

10. [DV92], [Mat96], [Drm97] Let $\tilde{A}^{(k)}$, $k = 0, 1, 2, \dots$ be the matrices computed by Algorithm 2 in floating point arithmetic. Write each $\tilde{A}^{(k)}$ as $\tilde{A}^{(k)} = B^{(k)} D^{(k)}$, with diagonals $D^{(k)}$ and $B^{(k)}$ with columns of unit Euclidean norms. Then

- $\tilde{A}^{(k+1)} = (\tilde{A}^{(k)} + \delta \tilde{A}^{(k)}) \check{V}^{(k)}$, where $\check{V}^{(k)}$ is the exact plane rotation transforming pivot columns, and $\delta \tilde{A}^{(k)}$ is zero except in the i_k th and the j_k th columns.
- $\tilde{A}^{(k)} + \delta \tilde{A}^{(k)} = (B^{(k)} + \delta B^{(k)}) D^{(k)}$, with $\|\delta B^{(k)}\|_F < c_k \epsilon$, where c_k is a small factor (e.g., $c_k < 20$) that depends on the implementation of the rotation.
- The above holds as long as the Euclidean norms of the i_k th and the j_k th columns of $\tilde{A}^{(k)}$ do not underflow or overflow. It holds even if the computed angle is so small that the computed value of $\tan \phi_k$ underflows (gradually or flushed to zero). This, however, requires special implementation of the Jacobi rotation.
- If all matrices $\tilde{A}^{(j)}$, $j = 0, \dots, k-1$ are nonsingular, then for all $i = 1, \dots, n$,

$$\max \left\{ 0, \prod_{j=0}^{k-1} (1 - \eta_j) \right\} \leq \frac{\sigma_i(\tilde{A}^{(k)})}{\sigma_i(A)} \leq \prod_{j=0}^{k-1} (1 + \eta_j),$$

where $\eta_j = \|\delta \tilde{A}^{(j)} (\tilde{A}^{(j)})^\dagger\|_2 \leq c_j \epsilon \|(B^{(j)})^\dagger\|_2$, and $\sigma_i(\cdot)$ stands for the i th largest singular value of a matrix. The accuracy of Algorithm 2 is determined by $\beta \equiv \max_{k \geq 0} \|(B^{(k)})^\dagger\|_2$. It is observed in practice that β never exceeds $\|B^\dagger\|_2$ too much.

- If A initially contains column-wise small uncertainty and if $\beta / \|B^\dagger\|_2$ is moderate, then Algorithm 2 computes as accurate approximations of the singular values as warranted by the data.

Examples:

1. Using orthogonal factorizations in finite precision arithmetic does not guarantee high relative accuracy in SVD computation. Bidiagonalization, as a preprocessing step in state-of-the-art SVD

algorithms, is an example. We use MATLAB with roundoff $\epsilon = \text{eps} \approx 2.22 \cdot 10^{-16}$. Let $\xi = 10/\epsilon$. Then floating point bidiagonalization of

$$A = \begin{pmatrix} 1 & 1 & \boxed{1} \\ 0 & 1 & \xi \\ 0 & -1 & \xi \end{pmatrix} \text{ yields } \tilde{A}^{(1)} = \begin{pmatrix} 1 & \alpha & 0 \\ 0 & \beta & \beta \\ 0 & \boxed{\beta} & \beta \end{pmatrix}, \quad \tilde{A}^{(2)} = \begin{pmatrix} 1 & \alpha & 0 \\ 0 & \gamma & \gamma \\ 0 & 0 & 0 \end{pmatrix},$$

$\alpha \approx 1.4142135\text{e}+0$, $\beta \approx 3.1845258\text{e}+16$, $\gamma \approx 4.5035996\text{e}+16$. The matrices $\tilde{A}^{(1)}$ and $\tilde{A}^{(2)}$ are computed using Givens plane rotations at the positions indicated by $\boxed{\cdot}$. The computed matrix $\tilde{A}^{(1)}$ entry-wise approximates with high relative accuracy the matrix $A^{(1)}$ from the exact computation (elimination of A_{13}). However, A and $A^{(1)}$ are nonsingular and $\tilde{A}^{(1)}$ is singular. The smallest singular value of A is lost — $\tilde{A}^{(1)}$ carries no information about σ_3 . Transformation from $\tilde{A}^{(1)}$ to $\tilde{A}^{(2)}$ is again perfectly entry-wise accurate. But, even exact SVD of the bidiagonal $\tilde{A}^{(2)}$ cannot restore the information lost in the process of bidiagonalization. (See Fact 4 above.)

On the other hand, let $\hat{A}^{(1)}$ denote the matrix obtained from A by perturbing A_{13} to zero. This changes the third column $\mathbf{a}_3$ of A by $\|\delta \mathbf{a}_3\|_2 \leq (\epsilon/14)\|\mathbf{a}_3\|_2$ and causes, at most, $\epsilon/5$ relative perturbation in the singular values. (See Fact 3 above. Here, $A = BD$ with $\kappa(B) < 2$.) If we apply Givens rotation from the left to annihilate the $(3, 2)$ entry in $\hat{A}^{(1)}$, the introduced perturbation is column-wise small. Also, the largest singular value decouples; the computed matrix is $\hat{A}^{(2)} = \begin{bmatrix} 1 & 1 \\ 0 & \alpha \end{bmatrix} \oplus \tilde{\sigma}_1$.

46.2 Preconditioned Jacobi SVD Algorithm

The accuracy properties and the run-time efficiency of the one-sided Jacobi SVD algorithm can be enhanced using appropriate preprocessing and preconditioning. More details can be found in [Drm94], [Drm99], [DV05a], and [DV05b].

Definitions:

QR factorization with column pivoting of $A \in \mathbb{R}^{m \times n}$ is the factorization $A\Pi = Q \begin{bmatrix} R \\ 0 \end{bmatrix}$, where Π is a permutation matrix, Q is orthogonal, and R is $n \times n$ upper triangular.

The QR factorization with **Businger–Golub pivoting** chooses Π to guarantee that $r_{kk}^2 \geq \sum_{i=k}^j r_{ij}^2$ for all $j = k, \dots, n$ and $k = 1, \dots, n$.

The QR factorization with **Powell–Reid’s complete (row and column) pivoting** computes the QR factorization using row and column permutations, $\Pi_r A \Pi_c = Q \begin{bmatrix} R \\ 0 \end{bmatrix}$, where Π_c is as in Businger–Golub pivoting and Π_r enhances numerical stability in case of A with differently scaled rows.

QR factorization with pivoting of A is **rank revealing** if small singular values of A are revealed by correspondingly small diagonal entries of R .

Facts:

1. [Drm94], [Hig96]. Let the QR factorization $A = Q \begin{bmatrix} R \\ 0 \end{bmatrix}$ of $A \in \mathbb{R}^{m \times n}$, $m \geq n$, be computed using the Givens or the Householder algorithm in the IEEE floating point arithmetic with rounding relative error $\epsilon < 10^{-7}$. Let the computed approximations of Q and R be $\tilde{Q}$ and $\tilde{R}$, respectively. Then there exist an orthogonal matrix $\hat{Q}$ and a backward perturbation δA such that

$$A + \delta A = \hat{Q} \begin{bmatrix} \tilde{R} \\ 0 \end{bmatrix}, \quad \|\tilde{Q}_{1:m,i} - \hat{Q}_{1:m,i}\|_2 \leq \epsilon_{qr}, \quad \|\delta A_{1:m,i}\|_2 \leq \epsilon_{qr} \|A_{1:m,i}\|_2$$

holds for all $i = 1, \dots, n$, with $\varepsilon_{qr} \leq O(mn)\epsilon$. This remains true if the factorization is computed with pivoting. It suffices to assume that the matrix A is already prepermuted on input, and that the factorization itself is performed without pivoting.

2. [Drm94], [Mat96], [Drm99] Let $A = BD$, where D is diagonal and B has unit columns in Euclidean norm.
 - If $\|B^\dagger\|_2$ is moderate, then the computed $\tilde{R}$ allows approximation of the singular values of A to high relative accuracy. If $\tilde{R}$ is computed with Businger–Golub pivoting, then Algorithm 2 applied to $\tilde{R}^T$ converges swiftly and computes the singular values of A with relative accuracy determined by $\|B^\dagger\|_2$.
 - Let A have full column rank and let its QR factorization be computed by Businger–Golub column pivoting. Write $R = ST$, where S is the diagonal matrix and T has unit rows in Euclidean norm. Then $\|T^{-1}\|_2 \leq n\|B^\dagger\|_2$.
3. [DV05a] The following algorithm carefully combines the properties of the one-sided Jacobi SVD and the properties of the QR factorization (Facts 1 and 2):

Algorithm 3: Preconditioned Jacobi SVD

$(U, \Sigma, [V]) = \text{SVD1}(A)$

Input: $A \in \mathbb{R}^{m \times n}$, $m \geq n$.

1: $(P_r A)P_c = Q_1 \begin{bmatrix} R_1 \\ 0 \end{bmatrix}$; % $R_1 \in \mathbb{R}^{\rho \times n}$, rank revealing QRF.

2: $R_1^T = Q_2 \begin{bmatrix} R_2 \\ 0 \end{bmatrix}$; % $R_2 \in \mathbb{R}^{\rho \times \rho}$.

3: $(U_2, \Sigma_2, V_2) = \text{SVD0}(R_2^T)$; % one-sided Jacobi SVD on R_2^T .

Output: $U = P_r^T Q_1 \begin{bmatrix} U_2 & 0 \\ 0 & I_{m-\rho} \end{bmatrix}$; $\Sigma = \begin{bmatrix} \Sigma_2 & 0 \\ 0 & 0 \end{bmatrix}$; $V = P_c Q_2 \begin{bmatrix} V_2 & 0 \\ 0 & I_{n-\rho} \end{bmatrix}$.

4. [Drm99], [DV05a], [DV05b]

- Let A and the computed matrix $\tilde{R}_1 \approx R_1$ in Algorithm 3 be of full column rank. Let $\tilde{R}_2$ be computed approximation of R_2 in Step 2. Then, there exist perturbations δA , $\delta \tilde{R}_1$, and there exist orthogonal matrices $\hat{Q}_1$, $\hat{Q}_2$ such that

$$(I + \delta A A^\dagger)A(I + \tilde{R}_1^{-1} \delta \tilde{R}_1) = \hat{Q}_1 \begin{bmatrix} \tilde{R}_2^T \\ 0 \end{bmatrix} \hat{Q}_2^T.$$

Thus, the first two steps of Algorithm 3 preserve the singular values if $\|B^\dagger\|_2$ is moderate.

- Let the one-sided Jacobi SVD algorithm with row cyclic pivot strategy be applied to $\tilde{R}_2^T$. Let the stopping criterion be satisfied at the matrix $(\tilde{R}_2^T)^{(k)}$ during the s th sweep. Write $(\tilde{R}_2^T)^{(k)} = \tilde{U}_2 \tilde{\Sigma}$, where $\tilde{\Sigma}$ is diagonal and $\tilde{U}_2$ is numerically orthogonal. Then there exist an orthogonal matrix $\hat{V}_2$ and a backward error $\delta \tilde{R}_2$ such that $\tilde{U}_2 \tilde{\Sigma} = (\tilde{R}_2 + \delta \tilde{R}_2)^T \hat{V}_2$, $\|\delta(\tilde{R}_2)_{1:\rho,i}\|_2 \leq \varepsilon_J \|\tilde{R}_2\|_{1:\rho,i}$, $i = 1, \dots, \rho$, with $\varepsilon_J \leq (1 + 6\epsilon)^{s(2n-3)} - 1$. If $\hat{U}_2$ is the closest orthogonal matrix to $\tilde{U}_2$, then $\tilde{R}_2^T = \hat{V}_2 \tilde{\Sigma} \hat{U}_2^T (I + E)$, where the dominant part of $\|E\|_2$ is $\|\delta \tilde{R}_2 \tilde{R}_2^{-1}\|_2$. Similar holds for any serial or parallel convergent pivot strategy.
- Assembling the above yields

$$(I + \delta A A^\dagger)A(I + \tilde{R}_1^{-1} \delta \tilde{R}_1) = \hat{Q}_1 \begin{bmatrix} \hat{V}_2 & 0 \\ 0 & I \end{bmatrix} \begin{bmatrix} \tilde{\Sigma} \\ 0 \end{bmatrix} \hat{U}_2^T \hat{Q}_2^T (I + \hat{Q}_2 E \hat{Q}_2^T).$$

The upper bound on the maximal relative error $\|\Sigma^{-1}(\Sigma - \tilde{\Sigma})\|_2$ is dominated by $\|\delta A A^\dagger\|_2 + \|\tilde{R}_1^{-1} \delta \tilde{R}_1\|_2 + \|\delta \tilde{R}_2 \tilde{R}_2^{-1}\|_2$.

- Let A have the structure $A = BD$, where full-column rank B has equilibrated columns and D is arbitrary diagonal scaling. The only relevant condition number for relative accuracy of Algorithm 3 is $\|B^\dagger\|_2$.
- 5. [DV05a], [DV05b] Algorithm 3 outperforms Algorithm 2 both in speed and accuracy.
- 6. [Drm99], [Drm00b], [Hig00], [DV05a] It is possible that $\kappa_2(B)$ is large, but A is structured as $A = D_1 C D_2$ with diagonal scalings D_1, D_2 (diagonal entries in nonincreasing order) and well conditioned C . In that case, desirable backward error for the first QR factorization (Step 1) is $P_r D_1 (C + \delta C) D_2 P_c$ with satisfactory bound on $\|\delta C\|_2$. This is nearly achieved if the QR factorization is computed with Powell–Reid’s complete pivoting or its simplification, which replaces row pivoting with initial sorting. Although theoretical understanding of this fact is not complete, initial row sorting (descending in $\|\cdot\|_\infty$ norm) is highly recommended.

Examples:

1. The key step of the preconditioning is in transposing the computed triangular factors. If A is a 100×100 Hilbert matrix, written as $A = BD$ (cf. Fact 2), then $\kappa_2(B) > 10^{19}$. If in Step 2 of Algorithm 3, we write R_1^T as $R_1^T = B_1 D_1$, where D_1 is diagonal and B_1 has unit columns in Euclidean norm, then $\kappa_2(B_1) < 50$.
2. Take $\gamma = 10^{-20}$, $\delta = 10^{-40}$ and consider the matrix

$$A = \begin{bmatrix} 1 & \gamma & \gamma \\ -\gamma & \gamma & \gamma^2 \\ 0 & \delta & 0 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & \gamma & 0 \\ 0 & 0 & \delta \end{bmatrix} \begin{bmatrix} 1 & \gamma & 1 \\ -1 & 1 & 1 \\ 0 & 1 & 0 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & \gamma \end{bmatrix},$$

with singular values nearly $\sigma_1 \approx 1$, $\sigma_2 \approx \gamma$, $\sigma_3 \approx 2\gamma\delta$, cf. [DGE99]. A cannot be written as $A = BD$ with diagonal D and well-conditioned B . Algorithm 2 computes no accurate digit of σ_3 , while Algorithm 3 approximates all singular values to nearly full accuracy. (See Fact 5.) The computed R_1^T in Step 2 is

$$\tilde{R}_1^T = \begin{bmatrix} -1.0000000000000000e + 0 & 0 & 0 \\ -1.0000000000000000e - 20 & -1.0000000000000000e - 20 & 0 \\ -1.0000000000000000e - 20 & -1.0000000000000000e - 40 & -2.0000000000000000e - 60 \end{bmatrix}.$$

Note that neither the columns of A nor the columns of $\tilde{R}_1$ reveal the singular value of order 10^{-60} . On the other hand, the Euclidean norms of the column of $\tilde{R}_1^T$ nicely approximate the singular values of A . (See Facts 2, 3.) If the order of the rows of A is changed, $\tilde{R}_1$ is computed with zero third row.

46.3 Accurate SVD from a Rank Revealing Decomposition: Structured Matrices

In some cases, the matrix A is given implicitly as the product of two or three matrices or it can be factored into such a product using more general nonorthogonal transformations. For instance, some specially structured matrices allow Gaussian eliminations with complete pivoting $P_1 A P_2 = L D U$ to compute entry-wise accurate $\tilde{L} \approx L$, $\tilde{D} \approx D$, $\tilde{U} \approx U$. Moreover, it is possible that the triple $\tilde{L}, \tilde{D}, \tilde{U}$ implicitly defines the SVD of A to high relative accuracy, and that direct application of any SVD algorithm directly to A does not return accurate SVD. For more information on matrices and bipartite graphs, see Chapter 30. For more information on sign pattern matrices, see Chapter 33. For more information on totally nonnegative (TN) matrices, see Chapter 21.

Definitions:

The singular values of A are said to be **perfectly well determined to high relative accuracy** if the following holds: No matter what the entries of the matrix are, changing an arbitrary nonzero entry $a_{k\ell}$ to $\theta a_{k\ell}$, with arbitrary $\theta \neq 0$, will cause perturbation $\sigma_i \rightsquigarrow \tilde{\sigma}_i \in [\theta_l \sigma_i, \theta_u \sigma_i]$, $\theta_l = \min\{|\theta|, 1/|\theta|\}$, $\theta_u = \max\{|\theta|, 1/|\theta|\}$, $i = 1, \dots, n$.

The **sparsity pattern** $\text{Struct}(A)$ of A is defined as the set of indices (k, ℓ) of the entries of A permitted to be nonzero.

The **bipartite graph** $\mathcal{G}(S)$ of the sparsity pattern S is the graph with vertices partitioned into row vertices $r_1, \dots, r_m$ and column vertices $c_1, \dots, c_n$, where r_k and r_l are connected if and only if $(k, l) \in S$.

If $\mathcal{G}(S)$ is acyclic, matrices with sparsity pattern S are **biacyclic**.

The **sign** pattern $\text{sgn}(A)$ prescribes locations and signs of nonzero entries of A .

A sign pattern S is **total signed compound** (TSC) if every square submatrix of every matrix A such that $\text{sgn}(A) = S$ is either **sign nonsingular** (nonsingular and determinant expansion is the sum of monomials of like sign) or **sign singular** (determinant expansion degenerates to sum of monomials, which are all zero); cf. Chapter 33.2.

$A \in \mathbb{R}^{m \times n}$ is **diagonally scaled totally unimodular** (DSTU) if there exist diagonal D_1, D_2 and **totally unimodular** Z (all minors of Z are from $\{-1, 0, 1\}$) such that $A = D_1 Z D_2$.

A decomposition $A = X D Y^T$ with diagonal matrix D is called an **RRD** if X and Y are full-column rank, well-conditioned matrices. (RRD is an abbreviation for “rank revealing decomposition,” but that term is defined slightly differently in Chapter 39.9, where X, Y are required to be orthogonal and D is replaced by an upper triangular matrix.)

Facts:

- [DG93] The singular values of A are perfectly well determined to high relative accuracy if and only if the bipartite graph $\mathcal{G}(S)$ is acyclic (forest of trees). Sparsity pattern S with acyclic $\mathcal{G}(S)$ allows at most $m + n - 1$ nonzero entries. A bisection algorithm computes all singular values of biacyclic matrices to high relative accuracy.
- [DK90], [FP94] Bidiagonal matrices are a special case of acyclic sparsity pattern. Let $n \times n$ bidiagonal matrix B be perturbed by $b_{ii} \rightsquigarrow b_{ii} \varepsilon_{2i-1}$, $b_{i,i+1} \rightsquigarrow b_{i,i+1} \varepsilon_{2i}$ for all admissible i s, and let $\varepsilon = \prod_{i=1}^{2n-1} \max\{|\varepsilon_i|, 1/|\varepsilon_i|\}$, where all $\varepsilon_i \neq 0$. If $\tilde{\sigma}_1 \geq \dots \geq \tilde{\sigma}_n$ are the singular values of the perturbed matrix, then for all i , $\sigma_i/\varepsilon \leq \tilde{\sigma}_i \leq \varepsilon \sigma_i$. The singular values of bidiagonal matrices are efficiently computed to high relative accuracy by the zero shift QR algorithm and the differential qd algorithm.
- [DGE99] Let $\mathcal{S}_\pm$ be a sparsity-and-sign pattern. Let each matrix A with pattern $\mathcal{S}_\pm$ have the property that small relative changes of its (nonzero) entries cause only small relative perturbations of its singular values. This is equivalent with $\mathcal{S}_\pm$ being total signed compound (TSC).
- [Drm98a], [DGE99] Let A be given by an RRD $A = X D Y^T$. Without loss of generality assume that X and Y have equilibrated (e.g., unit in Euclidean norm) columns. Algorithm 4 computes the SVD of A to high relative accuracy.

Algorithm 4: RRD Jacobi SVD

$(U, \Sigma, V) = \text{SVD2}(X, D, Y)$

Input: $X \in \mathbb{R}^{m \times p}$, $Y \in \mathbb{R}^{n \times p}$ full column rank, $D \in \mathbb{R}^{p \times p}$ diagonal.

1: $\Upsilon = Y D$; $\Upsilon P = Q \begin{bmatrix} R \\ 0 \end{bmatrix}$ % rank revealing QR factorization.

2: $Z = (X P) R^T$; % explicit standard matrix multiplication.

3: $(U_z, \Sigma_z, V_z) = \text{SVD1}(Z)$ % one-sided Jacobi SVD on Z .

Output: $U = U_z$; $\Sigma = \begin{bmatrix} \Sigma_z & 0 \\ 0 & 0 \end{bmatrix}$; $V = Q \begin{bmatrix} V_z & 0 \\ 0 & I_{n-p} \end{bmatrix}$. $X D Y^T = U \Sigma V^T$.

5. [Drm98a], [DGE99], [DM04]

- In Step 2 of Algorithm 4, the matrix R^T has the structure $R^T = L S$, where S is diagonal scaling, L has unit columns, and $\|L^{-1}\|_2$ is bounded by a function of p independent of input data. The upper bound on $\kappa_2(L)$ depends on pivoting P in Step 1 and can be $O(n^{1+(1/4)\log_2 n})$. For the Businger–Golub, pivoting the upper bound is $O(2^n)$, but in practice one can expect an $O(n)$ bound.
- The product $Z = (X P) R^T$ must not be computed by fast (e.g., Strassen) algorithm if all singular values are wanted to high relative accuracy. The reason is that fast matrix multiplication algorithms can produce larger component-wise perturbations than the standard $O(n^3)$ algorithm.
- The computed $\tilde{U}$, $\tilde{\Sigma}$, $\tilde{V}$ satisfy: There exist orthogonal $\hat{U} \approx \tilde{U}$, orthogonal $\hat{V} \approx \tilde{V}$, and E_1, E_2 such that $\hat{U} \tilde{\Sigma} \hat{V}^T = (I + E_1) A (I + E_2)$, $\|E_1\|_2 \leq O(\epsilon) \kappa_2(X)$, $\|E_2\|_2 \leq O(\epsilon) \kappa_2(L) \kappa_2(Y)$. The relative errors in the computed singular values are bounded by $O(\epsilon) \kappa_2(L) \max\{\kappa_2(X), \kappa_2(Y)\}$. Here, $O(\epsilon)$ denotes machine precision ϵ multiplied by a moderate polynomial in m, n, p .

6. [DGE99] Classes of matrices with accurate LDU factorization and, thus, accurate SVD computation by Algorithm 4, or other algorithms tailored for special classes of matrices (e.g., QR and qd algorithms for bidiagonal matrices), include:

- *Acyclic matrices*: Accurate LDU factorization with pivoting uses the correspondence between the monomials in determinant expansion and perfect matchings in $\mathcal{G}(S)$.
- *Total signed compound (TSC) matrices*: LDU factorization with complete pivoting $P_r A P_c = L D U$ of an TSC matrix A can be computed in a forward stable way. Cancellations in Gaussian eliminations can be avoided by computing some elements as quotients of minors. All entries of L, D, U are computed to high relative accuracy. The behavior of $\kappa(L)$ and $\kappa(U)$ is not fully analyzed, but they behave well in practice.
- *Diagonally scaled totally unimodular (DSTU) matrices*: This class includes acyclic and certain finite element (e.g., mass-spring systems) matrices. Gaussian elimination with complete pivoting is modified by replacing dangerous cancellations by exactly predicted exact zeros. All entries of L, D, U are computed to high relative accuracy, and $\kappa(L), \kappa(U)$ are at most $O(mn)$ and $O(n^2)$, respectively.

7. [Dem99] In some cases, new classes of matrices and accurate SVD computation are derived from relations with previously solved problems or by suitable matrix representations. The following cases are analyzed and solved with $O(n^3)$ complexity:

- *Cauchy matrices*: $C = C(x, y)$, $c_{ij} = 1/(x_i + y_j)$, and the parameters x_i s, y_i s are the initial data. The algorithm extends to Cauchy-like matrices $C' = D_1 C D_2$, where C is a Cauchy matrix and D_1, D_2 are diagonal scalings. The required LDU factorization is computed with $\frac{4}{3}n^3$ operations and given as input to Algorithm 4. It should be noted that this does not imply that Cauchy matrices determine their singular values perfectly well. The statement is: For a Cauchy matrix C , given by set of parameters x_i 's, y_i 's stored as floating point numbers in computer memory, all singular values of C can be computed in a forward stable way. Thus, for example, singular values of a notoriously ill-conditioned Hilbert matrix can be computed to high relative accuracy.
- *Vandermonde matrices*: $V = [v_{ij}]$, $v_{ij} = x_i^{j-1}$. An accurate algorithm exploits the fact that postmultiplication of V by the discrete Fourier transform matrix gives a Cauchy-like matrix. The finite precision arithmetic requires a guard digit and extra precision to tabulate certain constants.
- *Unit displacement rank matrix* X : Solution of matrix equation $AX + XB = d_1 d_2^T$, where A, B are diagonal, or normal matrices with known accurate spectral decompositions. The class of unit displacement rank matrices generalizes the Cauchy-like matrices (A and B diagonal) and Vandermonde matrices (A diagonal and B circular shift).

8. [DK04] Weakly diagonally dominant M-matrices given with $a_{ij} \leq 0$ for $i \neq j$ and with the row-sums $s_i = \sum_{j=1}^n a_{ij} \geq 0$, $i = 1, \dots, n$ known with small relative errors, allow accurate LDU factorization with complete pivoting.
9. [Koe05] Totally nonnegative (TN) matrix A can be expressed as the product of nonnegative bidiagonal matrices. If A is given implicitly by this bidiagonal representation, then all its singular values (and eigenvalues, too) can be computed from the bidiagonals to high relative accuracy. Accurate bidiagonal representation for given TN matrix A is possible for totally positive cases (all minors positive), provided certain pivotal minors can be computed accurately.
10. [DK05a] The singular values of the matrix V with entries $v_{ij} = P_i(x_j)$, where the P_i s are orthogonal polynomials and the x_j s are the nodes, can be computed to high relative accuracy.
11. [Drm00a] Accurate SVD of the RRD XDY^T extends to the triple product $XS Y^T$, where S is not necessarily diagonal, but it can be accurately factored by Gaussian eliminations with pivoting.

Examples:

1. An illustration of the power of the algorithms described in this section is the example of a 100×100 Hilbert matrix described in [Dem99]. Its singular values range over 150 orders of magnitude and are computed using the package Mathematica with 200-decimal digit software floating point arithmetic. The computed singular values are rounded to 16 digits and used as reference values. The singular values computed in IEEE double precision floating point ($\epsilon \approx 10^{-16}$) by the algorithms described in this section agree with the reference values with relative error less than $34 \cdot \epsilon$.
2. [DGE99] Examples of sign patterns of TSC matrices:

$$\begin{bmatrix} + & + & 0 & 0 & 0 \\ + & - & + & 0 & 0 \\ 0 & + & + & + & 0 \\ 0 & 0 & + & - & + \\ 0 & 0 & 0 & + & + \end{bmatrix}, \quad \begin{bmatrix} + & + & + & + & + \\ + & - & 0 & 0 & 0 \\ + & 0 & - & 0 & 0 \\ + & 0 & 0 & - & 0 \\ + & 0 & 0 & 0 & - \end{bmatrix}.$$

TSC matrices must be sparse because an $m \times n$ TSC matrix can have at most $\frac{3}{2}(m+n) - 2$ nonzero entries.

46.4 Positive Definite Matrices

Definitions:

The **Cholesky factorization with pivoting** of symmetric positive definite $n \times n$ matrix H computes the Cholesky factorization $P^T H P = L L^T$, where the permutation matrix P is such that $\ell_{ii}^2 \geq \sum_{k=i}^j \ell_{jk}^2$, $1 \leq i \leq j \leq n$.

The **component-wise relative distance** between H and its component-wise relative perturbation $\tilde{H}$ is $\text{reldist}(H, \tilde{H}) = \max_{i,j} \frac{|h_{ij} - \tilde{h}_{ij}|}{|h_{ij}|}$, where $0/0 = 0$.

A **diagonally scaled representation** of symmetric matrix H with positive diagonal entries is a factored representation $H = D A D$ with $D = \text{diag}(\sqrt{h_{11}}, \dots, \sqrt{h_{nn}})$, $a_{ij} = \frac{h_{ij}}{\sqrt{h_{ii} h_{jj}}}$.

Facts:

1. [Dem89], [Dem92] Let $H = D A D$ be a diagonally scaled representation of positive definite H , and let $\lambda_{\min}(A)$ denote the minimal eigenvalue of A .
 - If δH is a symmetric perturbation such that $H + \delta H$ is not positive definite, then

$$\max_{1 \leq i, j \leq n} \frac{|\delta h_{ij}|}{\sqrt{h_{ii} h_{jj}}} \geq \frac{\lambda_{\min}(A)}{n} = \frac{1}{n \|A^{-1}\|_2}.$$

- If $\delta H = -\lambda_{\min}(A)D^2$, then $\max_{i,j} \frac{|\delta h_{ij}|}{\sqrt{h_{ii}h_{jj}}} = \lambda_{\min}(A)$, $\text{reldist}(H, H + \delta H) = \lambda_{\min}(A)$, and $H + \delta H$ is singular.
2. [Dem89] Let $H = DAD$ be an $n \times n$ symmetric matrix with positive diagonal entries, stored in the machine memory. Let H be the input matrix in the Cholesky factorization algorithm. Then the following holds:
 - If the Cholesky algorithm successfully completes all operations and computes lower triangular matrix $\tilde{L}$, then there exists symmetric backward perturbation δH such that $\tilde{L}\tilde{L}^T = H + \delta H$ and $|\delta h_{ij}| \leq \eta_C \sqrt{h_{ii}h_{jj}}$, $\eta_C \leq O(n\epsilon)$.
 - If $\lambda_{\min}(A) > n\eta_C$, then the Cholesky algorithm will succeed and compute $\tilde{L}$.
 - If $\lambda_{\min}(A) < \epsilon$, then there exists simulation of rounding errors in which the Cholesky algorithm fails to complete all operations.
 - If $\lambda_{\min}(A) \leq -n\eta_C$, then it is certain that the Cholesky algorithm will fail.
 3. [DV92] If $H = DAD \in \text{PD}_n$ is perturbed to a positive definite $\tilde{H} = H + \delta H$, then
 - $\max_{1 \leq i \leq n} \frac{|\lambda_i - \tilde{\lambda}_i|}{\lambda_i} \leq \|L^{-1}\delta H L^{-T}\|_2 \leq \frac{\|D^{-1}\delta H D^{-1}\|_2}{\lambda_{\min}(A)} = \|A^{-1}\|_2 \|D^{-1}\delta H D^{-1}\|_2$.
 - Let $\delta H = \eta D^2$, with any $\eta \in (0, \lambda_{\min}(A))$. Then for some index i it holds that $\frac{|\lambda_i - \tilde{\lambda}_i|}{\lambda_i} \geq \sqrt[n]{1 + \frac{\eta}{\lambda_{\min}(A)}}$.
 4. [DV92], [VS93] Let $H = DAD$ be positive definite and $c > 0$ constant such that for all $\epsilon \in (0, 1/c)$ and for all symmetric component-wise relative perturbations δH with $|\delta h_{ij}| \leq \epsilon |h_{ij}|$, $1 \leq i, j \leq n$, the ordered eigenvalues λ_i and $\tilde{\lambda}_i$ of H and $H + \delta H$ satisfy $\max_{1 \leq i \leq n} \frac{|\lambda_i - \tilde{\lambda}_i|}{\lambda_i} \leq c\epsilon$. Then $\|A^{-1}\|_2 < (1+c)/2$. The same holds for more general perturbations, e.g., δH with $|\delta h_{ij}| \leq \epsilon \sqrt{h_{ii}h_{jj}}$.
 5. [vdS69] It holds that $\|A^{-1}\|_2 \leq \kappa_2(A) \leq n \min_{D=\text{diag}} \kappa_2(DHD) \leq n\kappa_2(H)$.
 6. For a general dense positive definite $H = DAD$ stored in the machine memory, eigenvalue computation with high relative accuracy is numerically feasible if and only if $\lambda_{\min}(A)$ is not smaller than the machine round-off unit. It is possible that matrix H is theoretically positive definite and that errors in computing its entries as functions of some parameters cause the stored matrix to be indefinite. Failure of the Cholesky algorithm is a warning that the matrix is entry-wise close to a symmetric matrix that is not positive definite.
 7. [VH89] If $P^T H P = L L^T$ is the Cholesky factorization with pivoting of positive definite H , then the SVD $L = U \Sigma V^T$ of L is computed very efficiently by the one-sided Jacobi SVD algorithm, and H is diagonalized as $H = (PU)\Sigma^2(PU)^T$.

Algorithm 5: Positive definite Jacobi EVD $(\lambda, U) = \text{EIG}_+(H)$ Input: $H \in \text{PD}_n$;

- 1: $P^T H P = L L^T$; % Cholesky factorization with pivoting.
- 2: **if** L computed successfully, **then**
- 3: $(U, \Sigma) = \text{SVD0}(L)$ % One-sided Jacobi SVD on L . V is not computed.
- 4: $\lambda_i = \Sigma_{ii}^2$, $U_{1:n,i} = P U_{1:n,i}$, $i = 1, \dots, n$;
- 5: Output: $\lambda = (\lambda_1, \dots, \lambda_n)$; U
- 6: **else**
- 7: **Error:** H is not numerically positive definite
- 8: **end if**

8. [DV92], [Mat96], [Drm98b]. Let $\tilde{\lambda}_1 \geq \cdots \geq \tilde{\lambda}_n$ be the approximations of the eigenvalues $\lambda_1 \geq \cdots \geq \lambda_n$ of $H = DAD$, computed by Algorithm 5. Then
 - The computed approximate eigenvalues of H are the exact eigenvalues of a nearby symmetric positive definite matrix $H + \delta H$, where $\max_{1 \leq i, j \leq n} \frac{|\delta h_{ij}|}{\sqrt{h_{ii}h_{jj}}} \leq \varepsilon$, and ε is bounded by $O(n)$ times the round-off unit ϵ .
 - $\max_{1 \leq i \leq n} \frac{|\lambda_i - \tilde{\lambda}_i|}{\lambda_i} \leq n\varepsilon \|A^{-1}\|_2$. The dominant part in the forward relative error is committed during the Cholesky factorization. The one-sided Jacobi SVD contributes to this error with, at most, $O(n)\epsilon\sqrt{\|A^{-1}\|_2} + O(n^2)\epsilon$.
9. Numerical properties of Algorithm 5, given in Fact 8, are better appreciated if compared with algorithms that first reduce H to tridiagonal matrix T and then diagonalize T . For such tridiagonalization based procedures the following hold:
 - The computed approximate eigenvalues of H are the exact eigenvalues of a nearby symmetric matrix $H + \delta H$, where $\|\delta H\|_2 \leq \varepsilon \|H\|_2$ and ε is bounded by a low degree polynomial in n times the round-off unit ϵ .
 - The computed eigenvalue approximations $\tilde{\lambda}_i \approx \lambda_i$ satisfy the *absolute error* bound $|\lambda_i - \tilde{\lambda}_i| \leq \varepsilon \|H\|_2$, that is, $\max_{1 \leq i \leq n} \frac{|\lambda_i - \tilde{\lambda}_i|}{\lambda_i} \leq \varepsilon \kappa_2(H)$.
10. In some applications it might be possible to work with a positive definite matrix implicitly as $H = C^T C$, where only a full column rank C is explicitly formed. Then the spectral computation with H is replaced with the SVD of C . The Cholesky factor L of H is computed implicitly from the QR factorization of C . This implicit formulation has many numerical advantages and it should be the preferred way of computation with positive definite matrices. An example is natural factor formulation of stiffness matrices in finite element computations.
11. [Drm98a], [Drm98b] Generalized eigenvalues of $HM - \lambda I$ and $H - \lambda M$ can be computed to high relative accuracy if $H = D_H A_H D_H$, $M = D_M A_M D_M$ with diagonal D_H , D_M and moderate $\|A_H^{-1}\|_2$, $\|A_M^{-1}\|_2$.

Examples:

1. In this numerical experiment we use MATLAB 6.5, Release 13 (on a Pentium 4 machine under MS Windows^R 2000), and the function `eig(.)` for eigenvalue computation. Let

$$H = \begin{bmatrix} 10^{40} & 10^{29} & 10^{19} \\ 10^{29} & 10^{20} & 10^9 \\ 10^{19} & 10^9 & 1 \end{bmatrix}.$$

The sensitivity of the eigenvalues of H and the accuracy of numerical algorithms can be illustrated by applications of the algorithms to various functions and perturbations of H . Let $P^T H P$ be obtained from H by permutation similarity with permutation matrix P . Let $H + \Delta H$ be obtained from H by changing H_{22} into $-H_{22} = -10^{20}$, and let $H + \delta H$ be obtained from H by multiplying H_{13} and H_{31} by $1 + \epsilon$, where $\epsilon \approx 2.22 \cdot 10^{-16}$ is the round-off unit in MATLAB. For the sake of experiment, the eigenvalues of numerically computed $(H^{-1})^{-1}$ are also examined. The values returned by `eig()` are shown in Table 46.1. All six approximations of the spectrum of H are with

small absolute error, $\max_{1 \leq i \leq 3} \frac{|\lambda_i - \tilde{\lambda}_i|}{\|H\|_2} \leq O(\epsilon)$. Some results might be different if a different version

of MATLAB or operating system is used.

2. Let H be the matrix from Example 1. H is positive definite, $\kappa_2(H) > 10^{40}$, and its $H = DAD$ representation with $D = \text{diag}(10^{20}, 10^{10}, 1)$ gives $\kappa_2(A) < 1.4$, $\|A^{-1}\|_2 < 1.2$. The Cholesky factor

TABLE 46.1

	$\text{eig}(H)$	$\text{eig}(P^T H P), P \simeq (2, 1, 3)$	$\text{eig}(\text{inv}(\text{inv}(H)))$
$\tilde{\lambda}_1$	1.000000000000000e + 040	1.000000000000000e + 040	1.000000000000000e + 040
$\tilde{\lambda}_2$	-8.100009764062724e + 019	9.900000000000000e + 019	9.900000000000000e + 019
$\tilde{\lambda}_3$	-3.966787845610502e + 023	9.818181818181818e - 001	9.818181818181817e - 001
	$\text{eig}(H + \Delta H)$	$\text{eig}(P^T H P), P \simeq (3, 2, 1)$	$\text{eig}(H + \delta H)$
$\tilde{\lambda}_1$	1.000000000000000e + 040	1.000000000000000e + 040	1.000000000000000e + 040
$\tilde{\lambda}_2$	-8.100009764062724e + 019	9.900000000000000e + 019	1.208844819952007e + 024
$\tilde{\lambda}_3$	-3.966787845610502e + 023	9.818181818181819e - 001	9.899993299416013e - 001

L of H is successfully computed in MATLAB by the `chol(·)` function. The matrix $L^T L$, which is implicitly diagonalized in Algorithm 5, reads

$$\hat{H} = L^T L = \begin{bmatrix} 1.000000000e + 40 & 9.949874371e + 18 & 9.908673886e - 02 \\ 9.949874371e + 18 & 9.900000000e + 19 & 8.962732759e - 02 \\ 9.908673886e - 02 & 8.962732759e - 02 & 9.818181818e - 01 \end{bmatrix}.$$

The diagonal of $\hat{H}$ approximates the eigenvalues of H with all shown digits correct. To see that, write $\hat{H}$ as $\hat{H} = \hat{D} \hat{A} \hat{D}$, where $\hat{D} = \text{diag}(\sqrt{\hat{H}_{11}}, \sqrt{\hat{H}_{22}}, \sqrt{\hat{H}_{33}})$ and

$$\hat{A} = \begin{bmatrix} 1.000000000e + 00 & 1.000000000e - 11 & 1.000000000e - 21 \\ 1.000000000e - 11 & 1.000000000e + 00 & 9.090909091e - 12 \\ 1.000000000e - 21 & 9.090909091e - 12 & 1.000000000e + 00 \end{bmatrix}$$

with $\|\hat{A} - I_3\| < 1.4 \cdot 10^{-11}$, $\|\hat{A}^{-1}\|_2 \approx 1$. Algorithm 5 computes the eigenvalues of H , as $\lambda_1 \approx 1.0e + 40$, $\lambda_2 \approx 9.900000000000002e + 19$, $\lambda_3 \approx 9.8181818181817e - 01$.

3. Smallest eigenvalues can be irreparably damaged simply by computing and storing matrix entries. This rather convincing example is discussed in [DGE99]. The stiffness matrix of a mass spring system with 3 masses,

$$K = \begin{bmatrix} k_1 + k_2 & -k_2 & 0 \\ -k_2 & k_2 + k_3 & -k_3 \\ 0 & -k_3 & k_3 \end{bmatrix}, \quad k_1, k_2, k_3 \text{ spring constants,}$$

is computed with $k_1 = k_3 = 1$ and $k_2 = \epsilon/2$, where ϵ is the round-off unit. Then the true and the computed assembled matrix are, respectively,

$$K = \begin{bmatrix} 1 + \epsilon/2 & -\epsilon/2 & 0 \\ -\epsilon/2 & 1 + \epsilon/2 & -1 \\ 0 & -1 & 1 \end{bmatrix}, \quad \tilde{K} = \begin{bmatrix} 1 & -\epsilon/2 & 0 \\ -\epsilon/2 & 1 & -1 \\ 0 & -1 & 1 \end{bmatrix}.$$

$\tilde{K}$ is the component-wise relative perturbation of K with $\text{reldist}(K, \tilde{K}) = \epsilon/(2 + \epsilon) < \epsilon/2$. K is positive definite with minimal eigenvalue near $\epsilon/4$, $\tilde{K}$ is indefinite with minimal eigenvalue near $-\epsilon^2/8$. MATLAB's function `chol(·)` fails to compute the Cholesky factorization of $\tilde{K}$ and reports that the matrix is not positive definite.

On the other hand, writing $K = A^T A$ with

$$A = \begin{bmatrix} \sqrt{k_1} & 0 & 0 \\ -\sqrt{k_2} & \sqrt{k_2} & 0 \\ 0 & -\sqrt{k_3} & \sqrt{k_3} \end{bmatrix} = \begin{bmatrix} \sqrt{k_1} & 0 & 0 \\ 0 & \sqrt{k_2} & 0 \\ 0 & 0 & \sqrt{k_3} \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 \\ -1 & 1 & 0 \\ 0 & -1 & 1 \end{bmatrix}$$

clearly separates physical parameters and the geometry of the connections. Since A is bidiagonal, for any choice of k_1, k_2, k_3 , the eigenvalues of K can be computed as squared singular values of A to nearly the same number of accurate digits to which the spring constants are given.

46.5 Accurate Eigenvalues of Symmetric Indefinite Matrices

Relevant relative perturbation theory for floating point computation with symmetric indefinite matrices is presented in [BD90], [DV92], [VS93], [DMM00], and [Ves00]. Full review of perturbation theory is given in Chapter 15.

Definitions:

Bunch–Parlett factorization of symmetric H is the factorization $P^T H P = L B L^T$, where P is a permutation matrix, B is a block-diagonal matrix with diagonal blocks of size 1×1 or 2×2 , and L is a full column rank unit lower triangular matrix, where the diagonal blocks in L that correspond to 2×2 blocks in B are 2×2 identity matrices.

A **symmetric rank revealing decomposition (SRRD)** of H is a decomposition $H = X D X^T$, where D is diagonal and X is a full column rank well-conditioned matrix.

Let $\mathcal{J}$ denote a nonsingular symmetric matrix. Matrix B is **$\mathcal{J}$ -orthogonal** if $F^T \mathcal{J} F = \mathcal{J}$. (Warning: this is a nonstandard usage of F , since in this book F usually denotes a field.)

The **hyperbolic SVD** decomposition of the matrix pair $(G, \mathcal{J})$ is a decomposition of G , $G = W \Sigma F^{-1}$, where W is orthogonal, Σ is diagonal, and F is $\mathcal{J}$ -orthogonal.

Facts:

1. If $H = U \Sigma V^T$ is the SVD of an $n \times n$ symmetric H , where $\Sigma = \bigoplus_{i=1}^m \sigma_{j_i} I_{n_i}$, then the matrix $V^T U$ is block-diagonal with m symmetric and orthogonal blocks of sizes $n_i \times n_i$, $i = 1, \dots, m$ along its diagonal. The n_i eigenvalues of the i th block are from $\{-1, 1\}$ and they give the signs of n_i eigenvalues of H with absolute value σ_{j_i} .
2. If $H = G \mathcal{J} G^T$ is a factorization with full column rank $G \in \mathbb{R}^{n \times r}$ and $\mathcal{J} = \text{diag}(\pm 1)$, then the eigenvalue problems $Hx = \lambda x$ and $(G^T G)y = \lambda \mathcal{J} y$ are equivalent. If F is the $\mathcal{J}$ -orthogonal eigenvector matrix of the pencil $G^T G - \lambda \mathcal{J}$ ($F^T \mathcal{J} F = \mathcal{J}$, $F^T (G^T G) F = \Sigma^2 = \text{diag}(\sigma_1^2, \dots, \sigma_r^2)$), then the matrix $\Sigma^2 \mathcal{J} = \text{diag}(\lambda_1, \dots, \lambda_r)$ contains the nonzero eigenvalues of H with the columns of $(GF) \Sigma^{-1}$ as corresponding eigenvectors.
3. [Ves00] Let H have factorization $H = G \mathcal{J} G^T$ as in Fact 2. Suppose that H is perturbed implicitly by changing $G \rightsquigarrow G + \delta G$, thus $\tilde{H} = (G + \delta G) \mathcal{J} (G + \delta G)^T$. Write $G = B D$, where D is diagonal and B has unit columns in the Euclidean norm, and let $\delta B = \delta G D^{-1}$. Let $\theta \equiv \|\delta B B^\dagger\|_2 < 1$. Then $\tilde{H}$ has the same number of zero eigenvalues as H and $\max_{\lambda_i \neq 0} \frac{|\delta \lambda_i|}{|\lambda_i|} \leq 2\theta + \theta^2$.
4. [Sla98], [Sla02] The factorization $H = G \mathcal{J} G^T$ in Fact 2 is computed by a modification of Bunch–Parlett factorization. Let $\tilde{G}$ be the computed factor and let $\tilde{\mathcal{J}} = \text{diag}(\pm 1)$ be the computed signature matrix. Then
 - A backward stability relation $H + \delta H = \tilde{G} \tilde{\mathcal{J}} \tilde{G}^T$ holds with the entry-wise bound $|\delta H| \leq O(n) \epsilon (|H| + |\tilde{G}| |\tilde{G}|^T)$.
 - Let $\tilde{G}$ have rank n . If $\tilde{\mathcal{J}} = \mathcal{J}$ and if $\hat{\lambda}_1 \geq \dots \geq \hat{\lambda}_n$ are the exact eigenvalues of the pencil $\tilde{G}^T \tilde{G} - \lambda \mathcal{J}$, then, for all i , $|\lambda_i - \hat{\lambda}_i| \leq \frac{O(n^2) \epsilon}{\sigma_{\min}^2(D^{-1} \tilde{G} \tilde{F})} |\lambda_i|$, where D denotes a diagonal matrix

such that $D^{-1} \tilde{G}$ has unit rows in Euclidean norm, $\tilde{F}$ is the eigenvector matrix of $\tilde{G}^T \tilde{G} - \lambda \mathcal{J}$, and $\sigma_{\min}(\cdot)$ denotes the minimal singular value of a matrix.

5. [Ves93] The one-sided (implicit) $\mathcal{J}$ -symmetric Jacobi algorithm essentially computes the hyperbolic SVD of $(G, \mathcal{J})$, $(W, \Sigma, F) = \text{HSVD}(G, \mathcal{J})$. It follows the structure of the one-sided Jacobi SVD (Algorithm 2) with the following modifications:

- On input, A is replaced with the pair $(G, \mathcal{J})$.
- In step k , $G^{(k+1)} = G^{(k)} V^{(k)}$ is computed from $G^{(k)}$ using Jacobi plane rotations exactly as in Algorithm 2 if $\mathcal{J}_{ii}$ and $\mathcal{J}_{jj}$ are of the same sign. (Here, $i = i_k$, $j = j_k$ are the pivot indices in $G^{(k)}$.)
- If $\mathcal{J}_{ii}$ and $\mathcal{J}_{jj}$ have opposite signs, then the Jacobi rotation is replaced with hyperbolic rotation

$$\begin{bmatrix} \cosh \zeta_k & \sinh \zeta_k \\ \sinh \zeta_k & \cosh \zeta_k \end{bmatrix}, \quad \tanh 2\zeta_k = -\frac{2\xi}{d_i + d_j},$$

$\xi = (G^{(k)})_{1:n,i}^T (G^{(k)})_{1:n,j}$, $d_\ell = (G^{(k)})_{1:n,\ell}^T (G^{(k)})_{1:n,\ell}$, $\ell = i, j$. The tangent is determined as $\tanh \zeta_k = \frac{\text{sign}(\tanh 2\zeta_k)}{|\tanh 2\zeta_k| + \sqrt{\tanh^2 2\zeta_k - 1}}$.

- The limit of $G^{(k)}$ is $W\Sigma$, and the accumulated product $V^{(0)} V^{(1)} \dots V^{(k)} \dots$ converges to $\mathcal{J}$ -orthogonal F , and $G = W\Sigma F^{-1}$ is the hyperbolic SVD of G .
 - The iterations are stopped at index k if $\max_{i \neq j} \frac{|(G^{(k)})_{1:n,i}^T (G^{(k)})_{1:n,j}|}{\|(G^{(k)})_{1:n,i}\|_2 \|(G^{(k)})_{1:n,j}\|_2} \leq \tau$. The tolerance τ is usually set to $n\epsilon$.
6. [Ves93] The eigenvalue problem of a symmetric indefinite matrix can be implicitly solved as a hyperbolic SVD problem. Algorithm 6 uses the factorization $H = G\mathcal{J}G^T$ (Fact 2) and hyperbolic SVD $\text{HSVD}(G, \mathcal{J})$ (Fact 5) to compute the eigenvalues and eigenvectors of H .

Algorithm 6: Hyperbolic Jacobi

$(\lambda, U) = \text{EIG0}(H)$

Input: $H \in \mathcal{S}_n$

1: $H = G\mathcal{J}G^T$, $\mathcal{J} = I_p \oplus (I_{n-p})$; % Bunch–Parlett factorization (modified).

2: $(W, \Sigma, F) = \text{HSVD}(G, \mathcal{J})$ % One-sided $\mathcal{J}$ -symmetric Jacobi algorithm.

3: $\lambda_i = \mathcal{J}_{ii} \cdot \Sigma_{ii}^2$; $U_{1:n,i} = W_{1:n,i}$, $i = 1, \dots, n$;

Output: $\lambda = (\lambda_1, \dots, \lambda_n)$; U

7. [Sla02] Let $\tilde{G}^{(0)} = \tilde{G}$, $\tilde{G}^{(k)}$, $k = 1, 2, \dots$ be the sequence of matrices computed by the one-sided $\mathcal{J}$ -symmetric Jacobi algorithm in floating point arithmetic. Write each $\tilde{G}^{(k)}$ as $\tilde{G}^{(k)} = B^{(k)} D^{(k)}$, where $D^{(k)}$ is the diagonal matrix with Euclidean column norms of $\tilde{G}^{(k)}$ along its diagonal.

- $\tilde{G}^{(k+1)}$ is the result of an exactly $\mathcal{J}$ -orthogonal plane transformation of $\tilde{G}^{(k)} + \delta \tilde{G}^{(k)} = (B^{(k)} + \delta B^{(k)}) D^{(k)}$, where $\|\delta B^{(k)}\|_F \leq c_k \epsilon$ with moderate factor c_k .

- Let $\tilde{\lambda}_1 \geq \dots \geq \tilde{\lambda}_n$ be computed as $\mathcal{J}_{ii}(\tilde{G}^{(k)})_{1:n,i}^T (\tilde{G}^{(k)})_{1:n,i}$, $i = 1, \dots, n$, where $\tilde{G}^{(k)}$ is the first matrix which satisfies stopping criterion. If k' is the index of last applied rotation, then

$$\max_{1 \leq i \leq n} \frac{|\hat{\lambda}_i - \tilde{\lambda}_i|}{|\hat{\lambda}_i|} \leq 2\eta + \eta^2, \text{ with } \hat{\lambda}_i \text{ as in Fact 4, and } \eta = O(\epsilon) \sum_{j=0}^{k'} \|(B^{(j)})^\dagger\|_2 + O(n^2)\epsilon + O(\epsilon^2).$$

8. [DMM03] Accurate diagonalization of indefinite matrices can be derived from their accurate SVD decomposition. (See Fact 1.)

Algorithm 7: Signed SVD
 $(\lambda, Q) = \text{EIG1}(H)$

 Input: $H \in \mathcal{S}_n$

- 1: $H = XDY^T$; % rank revealing decomposition.
 - 2: $(U, \Sigma, V) = \text{SVD2}(X, D, Y)$; % accurate SVD.
 - 3: Recover the signs of the eigenvalues, $\lambda_i = \pm\sigma_i$, using the structure of $V^T U$;
 - 4: Recover the eigenvector matrix Q using the structure of $V^T U$.
- Output: $\lambda = (\lambda_1, \dots, \lambda_n)$; Q

Let $H \approx \tilde{U} \tilde{\Sigma} \tilde{V}^T$, $\tilde{\Sigma} = \text{diag}(\tilde{\sigma}_1, \dots, \tilde{\sigma}_n)$, be the SVD computed by Algorithm 7.

- If all computed singular values $\tilde{\sigma}_i$, $i = 1, \dots, n$, are well separated, Algorithm 7 chooses $\tilde{\lambda}_i = \tilde{\sigma}_i \text{sign}(\tilde{V}_{1:n,i}^T \tilde{U}_{1:n,i})$, with the eigenvector $\tilde{Q}_{1:n,i} = \tilde{V}_{1:n,i}$.
 - If singular values are clustered, then clusters are determined by perturbation theory and the signs are determined inside each individual cluster.
9. [DMM03] If the rank revealing factorization in Step 1 of Algorithm 7 is computed as $H \approx \tilde{X} \tilde{D} \tilde{Y}^T$ with $\|\tilde{X} - X\| \leq \xi$, $\|\tilde{Y} - Y\| \leq \xi$, $\max_{i=1:n} |d_{ii} - \tilde{d}_{ii}|/|d_{ii}| \leq \xi$, then the computed eigenvalues $\tilde{\lambda}_i$ satisfy $|\lambda_i - \tilde{\lambda}_i| \leq \xi' \kappa(L) \max\{\kappa(X), \kappa(Y)\} |\lambda_i|$, $i = 1, \dots, n$. Here, ξ and ξ' are bounded by the round-off ϵ times moderate functions of the dimensions, and it is assumed that the traces of the diagonal blocks of $V^T U$ (Cf. Fact 1) are computed correctly.
 10. [DK05b] An accurate symmetric rank revealing decomposition $H = XDX^T$ can be given as input to Algorithm 7 or it can replace Step 1 in Algorithm 6 by defining $G = X\sqrt{|D|}$, $\mathcal{J} = \text{diag}(\text{sign}(d_{11}), \dots, \text{sign}(d_{nn}))$. Once an accurate SRRD is available, the eigenvalues are computed to high relative accuracy. For the following structured matrices, specially tailored algorithms compute accurate SRRDs:
 - Symmetric scaled Cauchy matrices, $H = DCD$, D diagonal, C Cauchy matrix.
 - Symmetric Vandermonde matrices, $V = (v^{(i-1)(j-1)})_{i,j=1}^n$, where $v \in \mathbb{R}$.
 - Symmetric totally nonnegative matrices.

Examples:

1. [Ves96] Initial factorization or rank revealing decomposition is the key for success or failure of the algorithms presented in this section. Let $\epsilon = 10^{-7}$ and

$$H = \begin{bmatrix} 1 & 1 & 1 \\ 1 & 0 & 0 \\ 1 & 0 & \epsilon \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 1 & \sqrt{\epsilon} \end{bmatrix} \begin{bmatrix} 0 & 1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 1 \\ 0 & 0 & \sqrt{\epsilon} \end{bmatrix}.$$

This factorization implies that all eigenvalues of H are determined to high relative accuracy. Whether or not they will be determined to that accuracy by Algorithm 6 (or any other algorithm depending on initial factorization) depends on the factorization. In Algorithm 6, the factorization in Step 1 chooses to start with 1×1 pivot, which after the first step gives

$$\begin{bmatrix} 1 & 0 & 0 \\ 1 & 1 & 0 \\ 1 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 \\ 0 & -1 & -1 \\ 0 & -1 & -1 + \epsilon \end{bmatrix} \begin{bmatrix} 1 & 1 & 1 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}.$$

The 2×2 Schur complement is ill-conditioned (entry-wise ϵ close to singularity, the condition number behaving as $1/\epsilon$) and the smallest eigenvalue is lost.

Author Note: This work is supported by the Croatian Ministry of Science, Education and Sports (Grant 0037120) and by a Volkswagen–Stiftung grant.

References

- [BD90] J. Barlow and J. Demmel. Computing accurate eigensystems of scaled diagonally dominant matrices. *SIAM J. Num. Anal.*, 27(3):762–791, 1990.
- [Dem89] J. Demmel. On floating point errors in Cholesky. LAPACK Working Note 14, Computer Science Department, University of Tennessee, October 1989.
- [Dem92] J. Demmel. The component-wise distance to the nearest singular matrix. *SIAM J. Matrix Anal. Appl.*, 13(1):10–19, 1992.
- [Dem99] J. Demmel. Accurate singular value decompositions of structured matrices. *SIAM J. Matrix Anal. Appl.*, 21(2):562–580, 1999.
- [DG93] J. Demmel and W. Gragg. On computing accurate singular values and eigenvalues of acyclic matrices. *Lin. Alg. Appl.*, 185:203–218, 1993.
- [DGE99] J. Demmel, M. Gu, S. Eisenstat, I. Slapničar, K. Veselić, and Z. Drmač. Computing the singular value decomposition with high relative accuracy. *Lin. Alg. Appl.*, 299:21–80, 1999.
- [DK90] J. Demmel and W. Kahan. Accurate singular values of bidiagonal matrices. *SIAM J. Sci. Stat. Comp.*, 11(5):873–912, 1990.
- [DK04] J. Demmel and P. Koev. Accurate SVDs of weakly diagonally dominant M-matrices. *Num. Math.*, 98:99–104, 2004.
- [DK05a] J. Demmel and P. Koev. Accurate SVDs of polynomial Vandermonde matrices involving orthonormal polynomials. *Lin. Alg. Appl.*, (to appear).
- [DK05b] F.M. Dopico and P. Koev. Accurate eigendecomposition of symmetric structured matrices. *SIAM J. Matrix Anal. Appl.*, (to appear).
- [DM04] F.M. Dopico and J. Moro. A note on multiplicative backward errors of accurate SVD algorithms. *SIAM J. Matrix Anal. Appl.*, 25(4):1021–1031, 2004.
- [DMM00] F.M. Dopico, J. Moro, and J.M. Molera. Weyl-type relative perturbation bounds for eigensystems of Hermitian matrices. *Lin. Alg. Appl.*, 309:3–18, 2000.
- [DMM03] F.M. Dopico, J.M. Molera, and J. Moro. An orthogonal high relative accuracy algorithm for the symmetric eigenproblem. *SIAM J. Matrix Anal. Appl.*, 25(2):301–351, 2003.
- [dR89] P.P.M. de Rijk. A one-sided Jacobi algorithm for computing the singular value decomposition on a vector computer. *SIAM J. Sci. Stat. Comp.*, 10(2):359–371, 1989.
- [Drm94] Z. Drmač. *Computing the Singular and the Generalized Singular Values*. Ph.D. thesis, Lehrgebiet Mathematische Physik, Fernuniversität Hagen, Germany, 1994.
- [Drm97] Z. Drmač. Implementation of Jacobi rotations for accurate singular value computation in floating point arithmetic. *SIAM J. Sci. Comp.*, 18:1200–1222, 1997.
- [Drm98a] Z. Drmač. Accurate computation of the product induced singular value decomposition with applications. *SIAM J. Numer. Anal.*, 35(5):1969–1994, 1998.
- [Drm98b] Z. Drmač. A tangent algorithm for computing the generalized singular value decomposition. *SIAM J. Num. Anal.*, 35(5):1804–1832, 1998.
- [Drm99] Z. Drmač. A *a posteriori* computation of the singular vectors in a preconditioned Jacobi SVD algorithm. *IMA J. Num. Anal.*, 19:191–213, 1999.
- [Drm00a] Z. Drmač. New accurate algorithms for singular value decomposition of matrix triplets. *SIAM J. Matrix Anal. Appl.*, 21(3):1026–1050, 2000.
- [Drm00b] Z. Drmač. On principal angles between subspaces of Euclidean space. *SIAM J. Matrix Anal. Appl.*, 22:173–194, 2000.
- [DV92] J. Demmel and K. Veselić. Jacobi’s method is more accurate than QR. *SIAM J. Matrix Anal. Appl.*, 13(4):1204–1245, 1992.
- [DV05a] Z. Drmač and K. Veselić. New fast and accurate Jacobi SVD algorithm: I. Technical report, Department of Mathematics, University of Zagreb, Croatia, June 2005. LAPACK Working Note 169.

- [DV05b] Z. Drmač and K. Veselić. New fast and accurate Jacobi SVD algorithm: II. Technical report, Department of Mathematics, University of Zagreb, Croatia, June 2005. LAPACK Working Note 170.
- [FP94] K.V. Fernando and B.N. Parlett. Accurate singular values and differential qd algorithms. *Num. Math.*, 67:191–229, 1994.
- [Hes58] M.R. Hestenes. Inversion of matrices by biorthogonalization and related results. *J. SIAM*, 6(1):51–90, 1958.
- [Hig96] N.J. Higham. *Accuracy and Stability of Numerical Algorithms*. SIAM, Philadelphia, 1996.
- [Hig00] N.J. Higham. QR factorization with complete pivoting and accurate computation of the SVD. *Lin. Alg. Appl.*, 309:153–174, 2000.
- [Jac46] C.G.J. Jacobi. Über ein leichtes Verfahren die in der Theorie der Säcularstörungen vorkommenden Gleichungen numerisch aufzulösen. *Crelle's Journal für Reine und Angew. Math.*, 30:51–95, 1846.
- [Koe05] P. Koev. Accurate eigenvalues and SVDs of totally nonnegative matrices. *SIAM J. Matrix Anal. Appl.*, 27(1):1–23, 2005.
- [Mat96] R. Mathias. Accurate eigensystem computations by Jacobi methods. *SIAM J. Matrix Anal. Appl.*, 16(3):977–1003, 1996.
- [Sla98] I. Slapničar. Component-wise analysis of direct factorization of real symmetric and Hermitian matrices. *Lin. Alg. Appl.*, 272:227–275, 1998.
- [Sla02] I. Slapničar. Highly accurate symmetric eigenvalue decomposition and hyperbolic SVD. *Lin. Alg. Appl.*, 358:387–424, 2002.
- [vdS69] A. van der Sluis. Condition numbers and equilibration of matrices. *Num. Math.*, 14:14–23, 1969.
- [Ves93] K. Veselić. A Jacobi eigenreduction algorithm for definite matrix pairs. *Num. Math.*, 64:241–269, 1993.
- [Ves96] K. Veselić. Note on the accuracy of symmetric eigenreduction algorithms. *ETNA*, 4:37–45, 1996.
- [Ves00] K. Veselić. Perturbation theory for the eigenvalues of factorised symmetric matrices. *Lin. Alg. Appl.*, 309:85–102, 2000.
- [VH89] K. Veselić and V. Hari. A note on a one-sided Jacobi algorithm. *Num. Math.*, 56:627–633, 1989.
- [VS93] K. Veselić and I. Slapničar. Floating point perturbations of Hermitian matrices. *Lin. Alg. Appl.*, 195:81–116, 1993.

Computational Linear Algebra

47	Fast Matrix Multiplication	<i>Dario A. Bini</i>	47-1
	Basic Concepts • Fast Algorithms • Other Algorithms • Approximation Algorithms • Advanced Techniques • Applications		
48	Structured Matrix Computations	<i>Michael Ng</i>	48-1
	Structured Matrices • Direct Toeplitz Solvers • Iterative Toeplitz Solvers • Linear Systems with Matrix Structure • Total Least Squares Problems		
49	Large-Scale Matrix Computations	<i>Roland W. Freund</i>	49-1
	Basic Concepts • Sparse Matrix Factorizations • Krylov Subspaces • The Symmetric Lanczos Process • The Nonsymmetric Lanczos Process • The Arnoldi Process • Eigenvalue Computations • Linear Systems of Equations • Dimension Reduction of Linear Dynamical Systems		

47

Fast Matrix Multiplication

Dario A. Bini
Università di Pisa

47.1	Basic Concepts	47-1
47.2	Fast Algorithms	47-2
47.3	Other Algorithms	47-5
47.4	Approximation Algorithms	47-6
47.5	Advanced Techniques	47-7
47.6	Applications	47-9
	References	47-11

Multiplying matrices is an important problem from both the theoretical and the practical point of view. Determining the arithmetic complexity of this problem, that is, the minimum number of arithmetic operations sufficient for computing an $n \times n$ matrix product, is still an open issue. Other important computational problems like computing the inverse of a nonsingular matrix, solving a linear system, computing the determinant, or more generally the coefficients of the characteristic polynomial of a matrix have a complexity related to that of matrix multiplication. Certain combinatorial problems, like the all pair shortest distance problem of a digraph, are strictly related to matrix multiplication. This chapter deals with fast algorithms for multiplication of unstructured matrices. Fast algorithms for structured matrix computations are presented in Chapter 48.

47.1 Basic Concepts

Let $A = [a_{i,j}]$, $B = [b_{i,j}]$, and $C = [c_{i,j}]$ be $n \times n$ matrices over the field F such that $C = AB$, that is, C is the matrix product of A and B .

Facts:

1. The elements of the matrices A , B , and C are related by the following equations:

$$c_{i,j} = \sum_{k=1}^n a_{i,k} b_{k,j}, \quad i, j = 1, \dots, n.$$

2. Each element $c_{i,j}$ is the scalar product of the i th row of A and the j th column of B and can be computed by means of n multiplications and $n - 1$ additions. This computation is described in Algorithm 1.
3. The overall cost of computing the n^2 elements of C is n^3 multiplications and $n^3 - n^2$ additions, that is $2n^3 - n^2$ arithmetic operations.

Algorithm 1. Conventional matrix multiplicationInput: the elements $a_{i,j}$ and $b_{i,j}$ of two $n \times n$ matrices A and B ;Output: the elements $c_{i,j}$ of the matrix product $C = AB$;**for** $i = 1$ **to** n **do** **for** $j = 1$ **to** n **do** $c_{i,j} = 0$ **for** $k = 1$ **to** n **do** $c_{i,j} = c_{i,j} + a_{i,k}b_{k,j}$

4. [Hig02, p. 71] The computation of $c_{i,j}$ by means of Algorithm 1 is element-wise forward numerically stable. More precisely, if $\tilde{C}$ denotes the matrix actually computed by performing Algorithm 1 in floating point arithmetic with machine precision ϵ , then $|C - \tilde{C}| \leq n\epsilon|A||B| + O(\epsilon^2)$, where $|A|$ denotes the matrix with elements $|a_{i,j}|$ and the inequality holds element-wise.

Examples:

- For 2×2 matrices Algorithm 1 requires 8 multiplications and 4 additions. For 5×5 matrices Algorithm 1 requires 125 multiplications and 100 additions.

47.2 Fast Algorithms

Definitions:

Define $\omega \in \mathbb{R}$ as the infimum of the real numbers w such that there exists an algorithm for multiplying $n \times n$ matrices with $O(n^w)$ arithmetic operations. ω is called the **exponent of matrix multiplication complexity**.

Algorithms that do not use commutativity of multiplication, are called **noncommutative algorithms**.

Algorithms for multiplying the $p \times p$ matrices A and B of the form

$$m_k = \left(\sum_{i,j=1}^p \alpha_{i,j,k} a_{i,j} \right) \left(\sum_{i,j=1}^p \beta_{i,j,k} b_{i,j} \right), \quad k = 1, \dots, t,$$

$$c_{i,j} = \sum_{k=1}^t \gamma_{i,j,k} m_k,$$

where $\alpha_{i,j,k}, \beta_{i,j,k}, \gamma_{i,j,k}$ are given scalar constants, are called **bilinear noncommutative algorithms** with t **nonscalar multiplications**.

Facts:

- [Win69] *Winograd's commutative algorithm*. For moderately large values of n , it is possible to compute the product of $n \times n$ matrices with less than $2n^3 - n^2$ arithmetic operations by means of the following simple identities where $n = 2m$ is even:

$$u_i = \sum_{k=1}^m a_{i,2k-1} a_{i,2k}, \quad v_i = \sum_{k=1}^m b_{2k-1,j} b_{2k,j}, \quad i = 1, n,$$

$$w_{i,j} = \sum_{k=1}^m (a_{i,2k-1} + b_{2k,j})(a_{i,2k} + b_{2k-1,j}), \quad i, j = 1, n,$$

$$c_{i,j} = w_{i,j} - u_i - v_j, \quad i, j = 1, n.$$

- The number of arithmetic operations required is $n^3 + 4n^2 - 2n$, which is less than $2n^3 - n^2$ already for $n \geq 8$. This formula, which for large values of n is faster by a factor of about 2 with respect to the

conventional algorithm, relies on the commutative property of multiplication. It can be extended to the case where n is odd.

3. [Str69] *Strassen's formula*. It is possible to multiply 2×2 matrices with only 7 multiplications instead of 8, but with a higher number of additions by means of the following identities:

$$\begin{aligned} m_1 &= (a_{1,2} - a_{2,2})(b_{2,1} + b_{2,2}), & m_5 &= a_{1,1}(b_{1,2} - b_{2,2}), \\ m_2 &= (a_{1,1} + a_{2,2})(b_{1,1} + b_{2,2}), & m_6 &= a_{2,2}(b_{2,1} - b_{1,1}), \\ m_3 &= (a_{1,1} - a_{2,1})(b_{1,1} + b_{1,2}), & m_7 &= (a_{2,1} + a_{2,2})b_{1,1}, \\ m_4 &= (a_{1,1} + a_{1,2})b_{2,2}, \\ c_{1,1} &= m_1 + m_2 - m_4 + m_6, & c_{1,2} &= m_4 + m_5, \\ c_{2,1} &= m_6 + m_7, & c_{2,2} &= m_2 - m_3 + m_5 - m_7. \end{aligned}$$

4. The overall number of arithmetic operations required by the Strassen formula is higher than the number of arithmetic operations required by the conventional matrix multiplication described in Algorithm 1 of section 47.1. However, the decrease from 8 to 7 of the number of multiplications provides important consequences.
5. The identities of Fact 3 do not exploit the commutative property of multiplication like the identities of Fact 1, therefore they are still valid if the scalar factors are replaced by matrices.
6. Strassen's formula provides a bilinear noncommutative algorithm where the constants $\alpha_{i,j,k}$, $\beta_{i,j,k}$, $\gamma_{i,j,k}$ are in the set $\{0, 1, -1\}$.
7. If n is even, say $n = 2m$, then A , B , and C can be partitioned into four $m \times m$ blocks, that is,

$$A = \begin{bmatrix} A_{1,1} & A_{1,2} \\ A_{2,1} & A_{2,2} \end{bmatrix}, \quad B = \begin{bmatrix} B_{1,1} & B_{1,2} \\ B_{2,1} & B_{2,2} \end{bmatrix}, \quad C = \begin{bmatrix} C_{1,1} & C_{1,2} \\ C_{2,1} & C_{2,2} \end{bmatrix},$$

where $A_{i,j}, B_{i,j}, C_{i,j} \in F^{m \times m}$, $i, j = 1, 2$, so that an $n \times n$ matrix product can be viewed as a 2×2 block matrix product. More specifically it holds that

$$\begin{aligned} C_{1,1} &= A_{1,1}B_{1,1} + A_{1,2}B_{2,1}, & C_{1,2} &= A_{1,1}B_{1,2} + A_{1,2}B_{2,2}, \\ C_{2,1} &= A_{2,1}B_{1,1} + A_{2,2}B_{2,1}, & C_{2,2} &= A_{2,1}B_{1,2} + A_{2,2}B_{2,2}. \end{aligned}$$

8. If $n = 2m$, the four blocks $C_{i,j}$ of Fact 7 can be computed by means of Strassen's formula with $7m \times m$ matrix multiplications and $18m \times m$ matrix additions, i.e., with $7(2m^3 - m^2) + 18m^2$ arithmetic operations. The arithmetic cost of matrix multiplication is reduced roughly by a factor of $7/8$.
9. [Str69] *Strassen's algorithm*. Furthermore, if m is even then the seven $m \times m$ matrix products of Fact 8 can be computed once again by means of Strassen's formula. If $n = 2^k$, k a positive integer, Strassen's formula can be repeated recursively until the size of the blocks is 1. Algorithm 2 synthesizes this computation.
10. The number $M(k)$ of arithmetic operations required by Algorithm 2 to multiply $2^k \times 2^k$ matrices is such that

$$\begin{aligned} M(k) &= 7M(k-1) + 18(2^{k-1})^2, \\ M(0) &= 1, \end{aligned}$$

which provides $M(k) = 7 \cdot 7^k - 6 \cdot 4^k = 7n^{\log_2 7} - 6n^2$, where $n = 2^k$ and $\log_2 7 = 2.8073 \dots < 3$. This yields the bound $\omega \leq \log_2 7$ on the exponent ω of matrix multiplication complexity.

11. In practice it is not convenient to carry out Strassen's algorithm up to matrices of size 1. In fact, for 2×2 matrices Strassen's formula requires much more operations than the conventional multiplication formula (see Example 1). Therefore, in the actual implementation the recursive iteration of Strassen's algorithm is stopped at size $p = 2^r$, where p is the largest value such that the conventional method applied to $p \times p$ matrices is faster than Strassen's method.

12. Strassen's algorithm can be carried out even though n is not an integer power of 2. Assume that $2^{k-1} < n < 2^k$, set $p = 2^k - n$, and embed the matrices A, B, C into matrices $\widehat{A}, \widehat{B}, \widehat{C}$ of size 2^k in the following way:

$$\widehat{A} = \begin{bmatrix} A & \mathbf{0}_{np} \\ \mathbf{0}_{pn} & \mathbf{0}_{pp} \end{bmatrix}, \quad \widehat{B} = \begin{bmatrix} B & \mathbf{0}_{np} \\ \mathbf{0}_{pn} & \mathbf{0}_{pp} \end{bmatrix}, \quad \widehat{C} = \begin{bmatrix} C & \mathbf{0}_{np} \\ \mathbf{0}_{pn} & \mathbf{0}_{pp} \end{bmatrix}.$$

Then one has $\widehat{C} = \widehat{A}\widehat{B}$ so that Strassen's algorithm can be applied to $\widehat{A}$ and $\widehat{B}$ in order to compute C . Even in this case the cost of Strassen's algorithm is still $O(n^{\log_2 7})$.

Algorithm 2. Strassen's algorithm

Procedure Strassen(k, A, B)

Input: the elements $a_{i,j}$ and $b_{i,j}$ of the $2^k \times 2^k$ matrices A and B ;

Output: the elements $c_{i,j}$ of the $2^k \times 2^k$ matrix $C = AB$;

If $k = 0$ **then**

output $c_{1,1} = a_{1,1}b_{1,1}$;

else

partition A, B , and C into $2^{k-1} \times 2^{k-1}$ blocks $A_{i,j}, B_{i,j}, C_{i,j}$, respectively,
where $i, j = 1, 2$;

compute

$$P_1 = A_{1,2} - A_{2,2}, \quad Q_1 = B_{2,1} + B_{2,2}, \quad P_5 = A_{1,1}, \quad Q_5 = B_{1,2} - B_{2,2},$$

$$P_2 = A_{1,1} + A_{2,2}, \quad Q_2 = B_{1,1} + B_{2,2}, \quad P_6 = A_{2,2}, \quad Q_6 = B_{2,1} - B_{1,1},$$

$$P_3 = A_{1,1} - A_{2,1}, \quad Q_3 = B_{1,1} + B_{1,2}, \quad P_7 = A_{2,1} + A_{2,2}, \quad Q_7 = B_{1,1},$$

$$P_4 = A_{1,1} + A_{1,2}, \quad Q_4 = B_{2,2},$$

for $i = 1$ **to** 7 **do**

$M_i = \text{Strassen}(k-1, P_i, Q_i)$;

compute

$$C_{1,1} = M_1 + M_2 - M_4 + M_6, \quad C_{1,2} = M_4 + M_5,$$

$$C_{2,1} = M_6 + M_7, \quad C_{2,2} = M_2 - M_3 + M_5 - M_7.$$

13. [Hig02, p. 440] *Numerical stability of Strassen's algorithm.* Let $\widetilde{C}$ be the $n \times n$ matrix obtained by performing Strassen's algorithm in floating point arithmetic with machine precision ϵ where $n = 2^k$. Then the following bound holds: $\max_{i,j} |\widetilde{c}_{i,j} - c_{i,j}| \leq \gamma_n \epsilon \max_{i,j} |a_{i,j}| \max_{i,j} |b_{i,j}| + O(\epsilon^2)$, where $\gamma_n = 6n^{\log_2 12}$ and $\log_2 12 \approx 3.585$. Thus, Strassen's algorithm has slightly less favorable stability properties than the conventional algorithm: the error bound does not hold component-wise but only norm-wise, and the multiplicative factor $6n^{\log_2 12}$ is larger than the factor n^2 of the bound given in Fact 4 of section 47.1.

Examples:

1. For $n = 16$, applying the basic Strassen algorithm to the 2×2 block matrices with blocks of size 8 and computing the seven products with the conventional algorithm requires $7 * (2 * 8^3 - 8^2) + 18 * 8^2 = 7872$ arithmetic operations. Using the conventional algorithm requires $2 * 16^3 - 16^2 = 7936$ arithmetic operations. Thus, it is convenient to stop the recursion of Algorithm 2 when $n = 16$. A similar analysis can be performed if the Winograd commutative formula of Fact 1 is used.

47.3 Other Algorithms

Facts:

1. [Win71] *Winograd's formula*. The following identities enable one to compute the product of 2×2 matrices with 7 multiplications and with 15 additions; this is the minimum number of additions among bilinear noncommutative algorithms for multiplying 2×2 matrices with 7 multiplications:

$$\begin{aligned} s_1 &= a_{2,1} + a_{2,2}, & s_2 &= s_1 - a_{1,1}, & s_3 &= a_{1,1} - a_{2,1}, & s_4 &= a_{1,2} - s_2, \\ t_1 &= b_{1,2} - b_{1,1}, & t_2 &= b_{2,2} - t_1, & t_3 &= b_{2,2} - b_{1,2}, & t_4 &= t_2 - b_{2,1}, \\ m_1 &= a_{1,1}b_{1,1}, & m_2 &= a_{1,2}b_{2,1}, & m_3 &= s_4b_{2,2}, & m_4 &= a_{2,2}t_4, \\ m_5 &= s_1t_1, & m_6 &= s_2t_2, & m_7 &= s_3t_3, \\ u_1 &= m_1 + m_6, & u_2 &= u_1 + m_7, & u_3 &= u_1 + m_5, \\ c_{1,1} &= m_1 + m_2, & c_{1,2} &= u_3 + m_3, \\ c_{2,1} &= u_2 - m_4, & c_{2,2} &= u_2 + m_5. \end{aligned}$$

The numerical stability of the recursive version of Winograd's formula is slightly inferior since the error bound of Fact 13 of Section 47.2 holds with $\gamma_n = 12n^{\log_2 18}$, $\log_2 18 \approx 4.17$.

2. [Win71], [BD78], [AS81] No algorithm exists for multiplying 2×2 matrices with less than 7 multiplications. The number of nonscalar multiplications needed for multiplying $n \times n$ matrices is at least $2n^2 - 1$.
3. If $n = 3^k$, the matrices A , B , and C can be partitioned into 9 blocks of size $n/3$ so that $n \times n$ matrix multiplication is reduced to computing the product of 3×3 block matrices. Formulas for 3×3 matrix multiplication that do not use the commutative property can be recursively used for general $n \times n$ matrix multiplication.
4. In general, if there exists a bilinear noncommutative formula for computing the product of $q \times q$ matrices with t nonscalar multiplications, then matrices of size $n = q^k$ can be multiplied with the cost of $O(q^t) = O(n^{\log_q t})$ arithmetic operations.
5. There exist algorithms for multiplying 3×3 matrices that require 25 multiplications [Gas71], 24 multiplications [Fid72], 23 multiplications [Lad76]. None of these algorithms beats Strassen's algorithm since $\log_3 21 < \log_2 7 < \log_3 22$. No algorithm is known for multiplying 3×3 matrices with less than 23 nonscalar multiplications.
6. [HM73] *Rectangular matrix multiplication and the duality property*. If there exists a bilinear noncommutative algorithm for multiplying two (rectangular) matrices of size $n_1 \times n_2$ and $n_2 \times n_3$, respectively, with t nonscalar multiplications, then there exist bilinear noncommutative algorithms for multiplying matrices of size $n_{\sigma_1} \times n_{\sigma_2}$ and $n_{\sigma_2} \times n_{\sigma_3}$ with t nonscalar multiplications for any permutation $\sigma = (\sigma_1, \sigma_2, \sigma_3)$.
7. If there exists a bilinear noncommutative algorithm for multiplying $n_1 \times n_2$ and $n_2 \times n_3$ matrices with t nonscalar multiplications, then square matrices of size $q = n_1 n_2 n_3$ can be multiplied with t^3 multiplications.
8. From Fact 7 and Fact 4 it follows that if there exists a bilinear noncommutative algorithm for multiplying $n_1 \times n_2$ and $n_2 \times n_3$ matrices with t nonscalar multiplications, then $n \times n$ matrices can be multiplied with $O(n^w)$ arithmetic operations, where $w = \log_{n_1 n_2 n_3} t^3$.
9. [HK71] There exist bilinear noncommutative algorithms for multiplying matrices of size 2×2 and 2×3 with 11 multiplications; there exist algorithms for multiplying matrices of size 2×3 and 3×3 with 15 multiplications.
10. There are several implementations of fast algorithms for matrix multiplication based on Strassen's formula and on Winograd's formula. In 1970, R. Brent implemented Strassen's algorithm on an IBM 360/67 (see [Hig02, p. 436]). This implementation was faster than the conventional algorithm already for $n \geq 110$. In 1988, D. Bailey provided a Fortran implementation for the Cray-2. Fortran codes, based on the Winograd variant have been provided since the late 1980s. For detailed comments and for more bibliography in this regard, we refer the reader to [Hig02, Sect. 23.3].

47.4 Approximation Algorithms

Matrices can be multiplied faster if we allow that the matrix product can be affected by some arbitrarily small nonzero error. Throughout this section, the underlying field F is $\mathbb{R}$ or $\mathbb{C}$ and we introduce a parameter $\lambda \in F$ that represents a nonzero number with small modulus. Multiplication by λ and λ^{-1} is negligible in the complexity estimate for two reasons: firstly, by choosing λ equal to a power of 2, multiplication by λ can be accomplished by shifting the exponent in the base-two representation of floating point numbers. This operation has a cost lower than the cost of multiplication. Secondly, in the block application of matrix multiplication algorithms, multiplication by λ corresponds to multiplying an $m \times m$ matrix by the scalar λ . This operation costs only m^2 arithmetic operations like matrix addition.

Definitions:

Algorithms for multiplying the $p \times p$ matrices A and B of the form

$$m_k = \left(\sum_{i,j=1}^p \alpha_{i,j,k} a_{i,j} \right) \left(\sum_{i,j=1}^p \beta_{i,j,k} a_{i,j} \right), \quad k = 1, \dots, t,$$

$$c_{i,j} = \sum_{k=1}^t \gamma_{i,j,k} m_k + \lambda p_{i,j}(\lambda),$$

where $\alpha_{i,j,k}, \beta_{i,j,k}, \gamma_{i,j,k}$ are given rational functions of λ and $p_{i,j}(\lambda)$ are polynomials, are called **Arbitrary Precision Approximating (APA) algorithms** with t **nonscalar multiplications** [Bin80], [BCL79].

Facts:

1. [BLR80] The matrix-vector product

$$\begin{bmatrix} f_1 \\ f_2 \end{bmatrix} = \begin{bmatrix} a & b \\ 0 & a \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} ax + by \\ ay \end{bmatrix}$$

cannot be computed with less than three multiplications. However, the following APA algorithm approximates f_1 and f_2 with two nonscalar multiplications:

$$m_1 = (a + \lambda b)(x + \lambda^{-1}y), \quad m_2 = ay,$$

$$f_1 \approx f_1 + \lambda bx = m_1 - \lambda^{-1}m_2,$$

$$f_2 = m_2.$$

The algorithm is not defined for $\lambda = 0$, but for $\lambda \rightarrow 0$ the output of the algorithm converges to the exact solution if performed in exact arithmetic.

2. [BCL79] Consider the 2×2 matrix product $C = AB$ where $a_{1,2} = 0$, i.e.,

$$\begin{bmatrix} c_{1,1} & c_{1,2} \\ c_{2,1} & c_{2,2} \end{bmatrix} = \begin{bmatrix} a_{1,1} & 0 \\ a_{2,1} & a_{2,2} \end{bmatrix} \begin{bmatrix} b_{1,1} & b_{1,2} \\ b_{2,1} & b_{2,2} \end{bmatrix}.$$

The elements $c_{i,j}$ can be approximated with 5 nonscalar multiplications by means of the following identities:

$$m_1 = (a_{1,1} + a_{2,2})(b_{1,2} + \lambda b_{2,1}), \quad m_2 = a_{1,1}(b_{1,1} + \lambda b_{2,1}), \quad m_3 = a_{2,2}(b_{1,2} + \lambda b_{2,2}),$$

$$m_4 = (a_{1,1} + \lambda a_{2,1})(b_{1,1} - b_{1,2}), \quad m_5 = (\lambda a_{2,1} - a_{2,2})b_{1,2},$$

$$c_{1,1} = m_2 - \lambda a_{1,1}b_{2,1}, \quad c_{1,2} = m_1 - m_3 - \lambda(a_{1,1}b_{2,1} + a_{2,2}b_{2,1} - a_{2,2}b_{2,2}),$$

$$c_{2,1} = \lambda^{-1}(m_1 - m_2 + m_4 + m_5), \quad c_{2,2} = \lambda^{-1}(m_3 + m_5).$$

3. Formulas of Fact 2 can be suitably adjusted to the case where only the element $a_{2,1}$ of the matrix A is zero.

4. The product of a 3×2 matrix and a 2×2 matrix can be approximated with 10 nonscalar multiplications by simply combining the formulas of Fact 2 and of Fact 3 in the following way:

$$\begin{bmatrix} \cdot & \cdot \\ \cdot & \cdot \\ \cdot & \cdot \end{bmatrix} = \left(\begin{bmatrix} \# & \# \\ 0 & \# \\ 0 & 0 \end{bmatrix} + \begin{bmatrix} 0 & 0 \\ \star & 0 \\ \star & \star \end{bmatrix} \right) \begin{bmatrix} \cdot & \cdot \\ \cdot & \cdot \end{bmatrix}.$$

5. Facts 4, 7, and 8 of section 47.3 are still valid for APA algorithms. By Fact 7 of Section 47.3 it follows that 1000 nonscalar multiplications are sufficient to approximate the product of 12×12 matrices.
6. By Fact 8 of Section 47.3 it follows that $O(n^{\log_{12} 1000})$ arithmetic operations are sufficient to approximate the product of $n \times n$ matrices, where $\log_{12} 1000 = 2.7798 \dots < \log_2 7$.
7. [BLR80] A rounding error analysis of an APA algorithm shows that the relative error in the output is bounded by $\alpha\lambda^h + \epsilon\beta\lambda^{-k}$, where α and β are positive constant depending on the input values, h and k are positive constants depending on the algorithm and ϵ is the machine precision. This bound grows to infinity as λ converges to zero. Asymptotically, in ϵ the minimum bound is $\gamma\epsilon^{\frac{h}{h+k}}$, for a constant γ .
8. [Bin80] *From approximate to exact computations*. Given an APA algorithm for approximating a $k \times k$ matrix product with t nonscalar multiplications, there exists an algorithm for $n \times n$ exact matrix multiplication requiring $O(n^w \log n)$ arithmetic operations where $w = \log_k t$. In particular, by Fact 5 an $O(n^w \log n)$ algorithm exists for exact matrix multiplication with $w = \log_{12} 1000$. This provides the bound $\omega \leq \log_{12} 1000$ for the exponent ω of matrix multiplication defined in Section 47.2.
9. The exact algorithm is obtained from the approximate algorithm by applying the approximate algorithm with $O(\log n)$ different values (not necessarily small) of λ and then taking a suitable linear combination of the $O(\log n)$ different values obtained in this way in order to get the exact product. More details on this approach, which is valid for any APA algorithm that does not use the commutative property of multiplication, can be found in [Bin80].

Examples:

1. The algorithm of Fact 1 computes $f_1(\lambda) = f_1 + \lambda bx$ that is close to f_1 for a small lambda. On the other hand, one has $f_1 = (f_1(1) + f_1(-1))/2$; i.e., a linear combination of the values computed by the APA algorithm with $\lambda = 1$ and with $\lambda = -1$ provides exactly f_1 .

47.5 Advanced Techniques

More advanced techniques have been introduced for designing fast algorithms for $n \times n$ matrix multiplication. The asymptotically fastest algorithm currently known requires $O(n^{2.38})$ arithmetic operations, but it is faster than the conventional algorithm only for huge values of n . Finding the infimum ω of the numbers w for which there exist $O(n^w)$ complexity algorithms is still an open problem. In this section, we provide a list of the main techniques used for designing asymptotically fast algorithms for $n \times n$ matrix multiplication.

Facts:

1. [Pan78] *Trilinear aggregating technique*. Different schemes for (approximate) matrix multiplication are based on the technique of trilinear aggregating by V. Pan. This technique is very versatile: Different algorithms based on this technique have been designed for fast matrix multiplication of several sizes [Pan84]; in particular, an algorithm for multiplying 70×70 matrices with 143,640 multiplications which leads to the bound $\omega < 2.795$.

2. [Sch81] *Partial matrix multiplication*. In the expression

$$c_{i,j} = \sum_{r=1}^m a_{i,r} b_{r,j}, \quad i, j = 1, \dots, m,$$

there are m^3 terms which are summed up. A partial matrix multiplication is encountered if some $a_{i,j}$ or some $b_{i,j}$ are zero, or if not all the elements $c_{i,j}$ are computed so that in the above expression there are less than m^3 terms, say, $k < m^3$. A. Schönhage [Sch81] has proved that if there exists a noncommutative bilinear (APA) algorithm for computing (approximating) partial $m \times m$ matrix multiplication with k terms that uses t nonscalar multiplications, then $\omega \leq 3 \log_k t$. This result applied to the formula of Fact 2 of Section 47.4 provides the bound $\omega \leq 3 \log_6 5 < 2.695$.

3. [Sch81] *Disjoint matrix multiplication* A. Schönhage has proved that it is possible to approximate two disjoint matrix products with less multiplications than the number of multiplications needed for computing separately these products. In particular, he has provided an APA algorithm for simultaneously multiplying a 4×1 matrix by a 1×4 and a 1×9 matrix by a 9×1 with only 17 nonscalar multiplications. Observe that 16 multiplications are *needed* for the former product and 9 multiplications are *needed* for the latter.
4. [Sch81] *The τ -theorem*. A. Schönhage has proven the τ -theorem. Namely, if the set of disjoint matrix products of size $m_i \times n_i$ times $n_i \times p_i$, for $i = 1, \dots, k$ can be approximated with t nonscalar multiplications by a bilinear noncommutative APA algorithm, then $\omega \leq 3\tau$, where τ solves the equation $\sum_{i=1}^k (m_i n_i p_i)^\tau = t$. From the disjoint matrix multiplication of Fact 3 it follows the equation $16^\tau + 9^\tau = 17$, which yields $\omega \leq 3\tau = 2.5479 \dots$
5. [Str87], [Str88] *Asymptotic spectrum: the laser method*. In 1988, V. Strassen introduced a powerful and sophisticated method, which, by taking tensor powers of set of bilinear forms that apparently are not completely related to matrix multiplication, provides some scheme for fast matrix multiplication. The name *laser method* was motivated from the fact that by tensor powering a set of “incoherent” bilinear forms it is possible to obtain a “coherent” set of bilinear forms.
6. *Lower bounds*. At least $2n^2 - 1$ multiplications are needed for multiplying $n \times n$ matrices by means of noncommutative algorithms [BD78]. If $n \geq 3$, then $2n^2 + n - 2$ multiplications are needed [Bla03]. The lower bound $\frac{5}{2}n^2 - 3n$ has been proved in [Bla01]. The nonlinear asymptotic lower bound $n^2 \log n$ has been proved in [Raz03]. At least $n^2 + 2n - 2$ multiplications are needed for approximating the product of $n \times n$ matrices by means of a noncommutative APA algorithm [Bin84]. The lower bound turns to $n^2 + \frac{3}{2}n - 2$ multiplications if commutativity is allowed. The product of 2×2 matrices can be approximated with 6 multiplications by means of a commutative algorithm [Bin84], 5 multiplications are needed. Seven multiplications are needed for approximating 2×2 matrix product by means of noncommutative algorithms [Lan05].
7. *History of matrix multiplication complexity*. After the 1969 paper by V. Strassen [Str69], where it was shown that $O(n^{\log_2 7})$ operations were sufficient for $n \times n$ matrix multiplication and inversion, the exponent ω of matrix multiplication complexity remained stuck at $2.807 \dots$ for almost 10 years until when V. Pan, relying on the technique of trilinear aggregating, provided a bilinear noncommutative algorithm for 70×70 matrix multiplication using 143,640 products. This led to the upper bound $\omega \leq \log_{70} 143640 \approx 2.795$. A few months later, Bini, Capovani, Lotti, and Romani [BCL79] introduced the concept of APA algorithms, and presented a scheme for approximating a 12×12 matrix product with 1000 products. In [Bin80] Bini showed that from any APA algorithm for matrix multiplication it is possible to obtain an exact algorithm with almost the same asymptotic complexity. This led to the bound $\omega \leq \log_{12} 1000 \approx 2.7798$. The technique of partial matrix multiplication was introduced by Schönhage [Sch81] in 1981 together with the τ -theorem, yielding the bound $\omega \leq 2.55$, a great improvement with respect to the previous estimates. This bound relies on the tools of trilinear aggregating and of approximate algorithms. Based on the techniques so far developed, V. Pan obtained the bound $\omega < 2.53$ in [Pan80] and one year later, F. Romani obtained the bound $\omega < 2.52$ in [Rom82]. The landmark bound $\omega < 2.5$ was obtained

TABLE 47.1 Main steps in the history of fast matrix multiplication

$\omega <$			
2.81	1969	[Str69]	Bilinear algorithms
2.79	1979	[Pan78]	Trilinear aggregating
2.78	1979	[BCL79],[Bin80]	Approximate algorithms
2.55	1981	[Sch81]	τ -theorem
2.53	1981	[Pan80]	
2.52	1982	[Rom82]	
2.50	1982	[CW72]	Refinement of the τ -theorem
2.48	1987,1988	[Str87],[Str88]	Laser method
2.38	1990	[CW82]	

by Coppersmith and Winograd [CW72] in 1982 by means of a refinement of the τ -theorem. In [Str87] Strassen introduced the powerful *laser method* and proved the bound $\omega < 2.48$. The laser method has been perfected by Coppersmith and Winograd [CW82], who proved the best estimate known so far, i.e., $\omega < 2.38$. Table 47.1 synthesizes this picture together with the main concepts used.

47.6 Applications

Some of the main applications of matrix multiplication are outlined in this section. For more details the reader is referred to [Pan84].

Definitions:

A square matrix A is **strongly nonsingular** if all its principal submatrices are nonsingular.

Facts:

1. *Classic matrix inversion.* Given a nonsingular $n \times n$ matrix A , the elements of A^{-1} can be computed by means of Gaussian elimination in $O(n^3)$ arithmetic operations.
2. *Inversion formula.* Let $n = 2m$ and partition the $n \times n$ nonsingular matrix A into four square blocks of size m as

$$A = \begin{bmatrix} A_{1,1} & A_{1,2} \\ A_{2,1} & A_{2,2} \end{bmatrix}.$$

Assume that $A_{1,1}$ is nonsingular. Denote $S = A_{2,2} - A_{2,1}A_{1,1}^{-1}A_{1,2}$ the Schur complement of $A_{1,1}$ in A and $R = A_{1,1}^{-1}$. Then S is nonsingular and the inverse of A can be written as

$$A^{-1} = \begin{bmatrix} R + RA_{1,2}S^{-1}A_{2,1}R & -RA_{1,2}S^{-1} \\ -S^{-1}A_{2,1}R & S^{-1} \end{bmatrix}.$$

Moreover, $\det A = \det S \det A_{1,1}$.

3. *Fast matrix inversion.* Let $n = 2^k$, with k positive integer and assume that A is strongly nonsingular. Then also $A_{1,1}$ and the Schur complement S are strongly nonsingular and the inversion formula of Fact 2 can be applied again to S by partitioning S into four square blocks of size $n/4$, recursively repeating this procedure until the size of the blocks is 1. The algorithm obtained in this way is described in Algorithm 3. Denoting by $\mathcal{I}(n)$ the complexity of this algorithm for inverting a strongly nonsingular $n \times n$ matrix and denoting by $\mathcal{M}(n)$ the complexity of the algorithm used

for $n \times n$ matrix multiplication, we obtain the expression

$$\begin{aligned}\mathcal{I}(n) &= 2\mathcal{I}(n/2) + 6\mathcal{M}(n/2) + n^2/2, \\ \mathcal{I}(1) &= 1.\end{aligned}$$

If $\mathcal{M}(n) = O(n^w)$ with $w \geq 2$, then one deduces that $\mathcal{I}(n) = O(n^w)$. That is, the complexity of matrix inversion is not asymptotically larger than the complexity of matrix multiplication.

4. The complexity of matrix multiplication is not asymptotically greater than the complexity of matrix inversion. This property follows from the simple identity

$$\begin{bmatrix} I & A & 0 \\ 0 & I & B \\ 0 & 0 & I \end{bmatrix}^{-1} = \begin{bmatrix} I & -A & AB \\ 0 & I & -B \\ 0 & 0 & I \end{bmatrix}.$$

5. Combining Facts 3 and 4, we deduce that matrix multiplication and matrix inversion have the same asymptotical complexity.

Algorithm 3: Fast matrix inversion

Procedure Fast_Inversion(k, A)

Input: the elements $a_{i,j}$ of the $2^k \times 2^k$ strongly nonsingular matrix A ;

Output: the elements $b_{i,j}$ of $B = A^{-1}$.

If $k = 0$, **then**

output $b_{1,1} = a_{1,1}^{-1}$;

else

partition A , into $2^{k-1} \times 2^{k-1}$ blocks $A_{i,j}$, $i, j = 1, 2$;

set $R = \text{Fast_Inversion}(k-1, A_{1,1})$;

compute $S = A_{2,2} - A_{2,1}RA_{1,2}$ and $V = \text{Fast_Inversion}(k-1, S)$

output

$$\begin{bmatrix} R + RA_{1,2}VA_{2,1}R & -RA_{1,2}V \\ -VA_{2,1}R & V \end{bmatrix}$$

6. The property of strong singularity for A is not a great restriction if $F = \mathbb{R}$ or $F = \mathbb{C}$. In fact, if A is nonsingular then $A^{-1} = (A^*A)^{-1}A^*$ and the matrix A^*A is strongly nonsingular.
7. *Computing the determinant.* Let A be strongly nonsingular. From Fact 2, $\det A = \det S \det A_{1,1}$. Therefore, an algorithm for computing $\det A$ can be recursively designed by computing S and then by applying recursively this algorithm to S and to $A_{1,1}$ until the size of the blocks is 1. Denoting by $\mathcal{D}(n)$ the complexity of this algorithm one has

$$\mathcal{D}(n) = 2\mathcal{D}(n/2) + \mathcal{I}(n/2) + 2\mathcal{M}(n/2) + n^2/4,$$

hence, if $\mathcal{M}(n) = O(n^w)$ with $w \geq 2$, then $\mathcal{D}(n) = O(n^w)$. In [BS83] it is shown that $\mathcal{M}(n) = O(\mathcal{D}(n))$

8. *Computing the characteristic polynomial.* The coefficients of the characteristic polynomial $p(x) = \det(A - xI)$ of the matrix A can be computed with the same asymptotic complexity of matrix multiplication.
9. *Combinatorial problems.* The complexity of some combinatorial problems is related to matrix multiplication, in particular, the complexity of the *all pair shortest distance problem* of finding the shortest distances $d(i, k)$ from i to k for all pairs (i, k) of vertices of a given digraph. We refer the reader to section 18 of [Pan84] for more details. The problem of Boolean matrix multiplication can be reduced to that of general matrix multiplication.

References

- [AS81] A. Alder and V. Strassen. On the algorithmic complexity of associative algebras. *Theoret. Comput. Sci.*, 15(2):201–211, 1981.
- [BS83] W. Baur and V. Strassen. The complexity of partial derivatives. *Theoret. Comput. Sci.*, 22(3):317–330, 1983.
- [Bin84] D. Bini. On commutativity and approximation. *Theoret. Comput. Sci.*, 28(1-2):135–150, 1984.
- [Bin80] D. Bini. Relations between exact and approximate bilinear algorithms. Applications. *Calcolo*, 17(1):87–97, 1980.
- [BCL79] D. Bini, M. Capovani, G. Lotti, and F. Romani. $O(n^{2.7799})$ complexity for $n \times n$ approximate matrix multiplication. *Inform. Process. Lett.*, 8(5):234–235, 1979.
- [BLR80] D. Bini, G. Lotti, and F. Romani. Approximate solutions for the bilinear forms computational problem. *SIAM J. Comp.*, 9:692–697, 1980.
- [Bla03] M. Bläser. On the complexity of the multiplication of matrices of small formats. *J. Complexity*, 19(1):43–60, 2003.
- [Bla01] M. Bläser. A $\frac{5}{2}n^2$ -lower bound for the multiplicative complexity of $n \times n$ -matrix multiplication. Lecture Notes in Comput. Sci., 2010, 99–109, Springer, Berlin, 2001.
- [BD78] R.W. Brockett and D. Dobkin. On the optimal evaluation of a set of bilinear forms. *Lin. Alg. Appl.*, 19(3):207–235, 1978.
- [CW82] D. Coppersmith and S. Winograd. On the asymptotic complexity of matrix multiplication. *SIAM J. Comp.*, 11(3):472–492, 1982.
- [CW72] D. Coppersmith and S. Winograd. Matrix multiplication via arithmetic progressions. *J. Symb. Comp.*, 9(3):251–280, 1972.
- [Fid72] C.M. Fiduccia. On obtaining upper bounds on the complexity of matrix multiplication. In R.E. Miller and J.W. Thatcher, Eds., *Complexity of Computer Computations*. Plenum Press, New York, 1972.
- [Gas71] N. Gastinel. Sur le calcul des produits de matrices. *Numer. Math.*, 17:222–229, 1971.
- [Hig02] N.J. Higham. *Accuracy and Stability of Numerical Algorithms*. Society for Industrial and Applied Mathematics (SIAM), Philadelphia, PA, 2nd ed., 2002.
- [HM73] J. Hopcroft and J. Musinski. Duality applied to the complexity of matrix multiplication and other bilinear forms. *SIAM J. Comp.*, 2:159–173, 1973.
- [HK71] J.E. Hopcroft and L.R. Kerr. On minimizing the number of multiplications necessary for matrix multiplication. *SIAM J. Appl. Math.*, 20:30–36, 1971.
- [Kel85] W. Keller-Gehrig. Fast algorithms for the characteristic polynomial. *Theor. Comp. Sci.*, 36:309–317, 1985.
- [Lad76] J.D. Laderman. A noncommutative algorithm for multiplying 3×3 matrices using 23 multiplications. *Bull. Amer. Math. Soc.*, 82(1):126–128, 1976.
- [Lan05] J.M. Landsberg. The border rank of the multiplication of 2×2 matrices is seven. *J. Am. Math. Soc.* (Electronic, September 26, 2005).
- [Pan84] V. Pan. How to multiply matrices faster, Vol. 179 of Lecture Notes in Computer Science. Springer-Verlag, Berlin, 1984.
- [Pan78] V. Ya. Pan. Strassen’s algorithm is not optimal. Trilinear technique of aggregating, uniting and canceling for constructing fast algorithms for matrix operations. In *19th Annual Symposium on Foundations of Computer Science (Ann Arbor, MI., 1978)*, pp. 166–176. IEEE, Long Beach, CA, 1978.
- [Pan80] V. Ya. Pan. New fast algorithms for matrix operations. *SIAM J. Comp.*, 9(2):321–342, 1980.
- [Raz03] R. Raz. On the complexity of matrix product. *SIAM J. Comp.*, 32(5):1356–1369, 2003.
- [Rom82] F. Romani. Some properties of disjoint sums of tensors related to matrix multiplication. *SIAM J. Comp.*, 11(2):263–267, 1982.
- [Sch81] A. Schönhage. Partial and total matrix multiplication. *SIAM J. Comp.*, 10(3):434–455, 1981.
- [Str69] V. Strassen. Gaussian elimination is not optimal. *Numer. Math.*, 13:354–356, 1969.

- [Str87] V. Strassen. Relative bilinear complexity and matrix multiplication. *J. Reine Angew. Math.*, 375/376:406–443, 1987.
- [Str88] V. Strassen. The asymptotic spectrum of tensors. *J. Reine Angew. Math.*, 384:102–152, 1988.
- [Win69] S. Winograd. The number of multiplications involved in computing certain functions. In *Information Processing 68 (Proc. IFIP Congress, Edinburgh, 1968)*, Vol. 1: Mathematics, Software, pages 276–279. North-Holland, Amsterdam, 1969.
- [Win71] S. Winograd. On multiplication of 2×2 matrices. *Lin. Alg. Appl.*, 4:381–388, 1971.

48

Structured Matrix Computations

Michael Ng

Hong Kong Baptist University

48.1	Structured Matrices	48-1
48.2	Direct Toeplitz Solvers.....	48-4
48.3	Iterative Toeplitz Solvers.....	48-5
48.4	Linear Systems with Matrix Structure	48-5
48.5	Total Least Squares Problems	48-8
	References	48-9

48.1 Structured Matrices

In various application fields, the matrices encountered have special structures that can be exploited to facilitate the solution process. Sparsity is one of these features. However, the matrices we consider in this chapter are mostly dense matrices with a special structure. Structured matrices have been around for a long time and are encountered in various fields of application. (See [GS84, KS98, BTY01].) Some interesting families are listed below. For simplicity, we give the definitions for real square matrices of size n .

Definitions:

Toeplitz matrices: Matrices with constant diagonals, i.e., $[T]_{i,j} = t_{i-j}$ for all $1 \leq i, j \leq n$:

$$T = \begin{bmatrix} t_0 & t_{-1} & \cdots & t_{2-n} & t_{1-n} \\ t_1 & t_0 & t_{-1} & & t_{2-n} \\ \vdots & t_1 & t_0 & \ddots & \vdots \\ t_{n-2} & & \ddots & \ddots & t_{-1} \\ t_{n-1} & t_{n-2} & \cdots & t_1 & t_0 \end{bmatrix}.$$

(See Chapter 16.2 for additional information on families of Toeplitz matrices.)

Lower shift matrix: The matrix with ones on the first subdiagonal and zeros elsewhere:

$$Z_n = \begin{bmatrix} 0 & 0 & \cdots & 0 & 0 \\ 1 & 0 & 0 & & 0 \\ \vdots & 1 & 0 & \ddots & \vdots \\ 0 & & \ddots & \ddots & 0 \\ 0 & 0 & \cdots & 1 & 0 \end{bmatrix}.$$